

Grembi Lab Manual

Updated: 2025-07-21

Contents

1	Welcome to the Grembi Lab!	7
1.1	About the lab	7
1.2	About this lab manual	7
2	Culture and conduct	9
2.1	Lab culture	9
2.2	Protecting human subjects	9
2.3	Authorship	10
2.4	Responsible use of AI tools	10
3	Communication and coordination	13
3.1	Slack	13
3.2	Email	14
3.3	Trello	14
3.4	Sharepoint	14
3.5	Meetings	14
4	Lab Safety	15
4.1	Key points regarding safety:	15
4.2	Required Institutional Lab Safety Training	16
4.3	Lab Safety Tour and site-specific hazard identification	16
4.4	Lab Safety Officer Responsibilities	17
5	Reproducibility	19
5.1	What is the reproducibility crisis?	19
5.2	Study design	20
5.3	Register study protocols	20
5.4	Write and register pre-analysis plans	20
5.5	Create reproducible workflows	21
5.6	Process and analyze data with internal replication and masking	21
5.7	Use reporting checklists with manuscripts	21
5.8	Publish preprints	21
5.9	Publish data (when possible) and replication scripts	22

6	Code repositories	23
6.1	Project Structure	23
6.2	.Rproj files	24
6.3	Configuration ('config') File	25
6.4	Order Files and Directories	25
6.5	Using Bash scripts to ensure reproducibility	25
7	Coding practices	27
7.1	Organizing scripts	27
7.2	Documenting your code	27
7.3	Object naming	29
7.4	Function calls	30
7.5	The here package	30
7.6	Reading/Saving Data	31
7.7	Integrating Box and Dropbox	31
7.8	Tidyverse	32
7.9	Coding with R and Python	34
7.10	Repeating analyses with different variations	34
7.11	Reviewing Code	38
8	Coding style	41
8.1	Comments	41
8.2	Line breaks	42
8.3	Automated Tools for Style and Project Workflow	44
9	Working with Big Data	47
9.1	The data.table package	47
9.2	Using downsampled data	48
9.3	Optimal RStudio set up	48
10	Data masking	49
10.1	General overview	49
10.2	Technical Overview	51
11	Github	55
11.1	Basics	55
11.2	Github Desktop	55
11.3	Git Branching	55
11.4	Example Workflow	56
11.5	Commonly Used Git Commands	57
11.6	How often should I commit?	58
11.7	What should be pushed to Github?	58
12	Unix commands	59
12.1	Basics	59
12.2	Syntax for both Mac/Windows	59
12.3	Running Bash Scripts	60

12.4 Running Rscripts in Windows	60
12.5 Checking tasks and killing jobs	61
12.6 Running big jobs	63
13 Reproducible Environments	67
13.1 Package Version Control with renv	67
14 Code Publication	69
14.1 Checklist overview	69
14.2 Fill out file headers	69
14.3 Clean up comments	69
14.4 Document functions	70
14.5 Remove deprecated filepaths	70
14.6 Ensure project runs via bash	70
14.7 Complete the README	70
14.8 Clean up feature branches	72
14.9 Create Github release	72
15 Data Publication	73
15.1 Overview	73
15.2 Removing PHI	74
15.3 Create public IDs	75
15.4 Create a data repository	77
15.5 Edit and test analysis scripts	77
15.6 Create a public GitHub page for public scripts	77
15.7 Go live	78
16 Slurm and cluster computing on ROAR	79
16.1 Getting started	79
16.2 Moving files to ROAR Collab	81
16.3 Installing packages on Sherlock	82
16.4 Testing your code	83
16.5 Storage & group storage access	85
16.6 Running big jobs	86
17 Checklists	89
17.1 Pre-analysis plan checklist	89
17.2 Code checklist	89
17.3 Manuscript checklist	90
17.4 Figure checklist	91
18 Resources	93
18.1 Resources for R	93
18.2 Resources for Git & Github Specifically tailored for R users . . .	94
18.3 Scientific figures	94
18.4 Writing	94
18.5 Presentations	94

18.6 Professional advice	94
18.7 Funding	94
18.8 Ethics and global health research	94

Chapter 1

Welcome to the Grembi Lab!

1.1 About the lab

Welcome to the lab of Dr. Jess Grembi, Assistant Professor of Pharmacology and member of the Huck Institutes of the Life Sciences at Penn State University. Our mission is to improve child and maternal health in low- and middle-income countries by understanding the mechanisms underlying environmental enteric dysfunction. Specifically, we study the microbiota that reside in the human small and large intestine, the metabolites they produce, and how these interact with the host immune system. Our focus is on improving the health of vulnerable populations from low-resource settings, particularly with respect to nutritional status. We are an interdisciplinary team and work collaboratively with microbiologists, immunologists, statisticians, epidemiologists, clinicians, and other scientists, both locally and globally. Together we tackle pressing questions related to child and maternal health, striving to make a meaningful impact in the communities we serve. To learn more about the lab, visit the lab website.

1.2 About this lab manual

This lab manual covers our communication strategy, code of conduct, laboratory safety, and best practices for reproducibility of computational workflows. It is a living document that is updated regularly.

This manual draws heavily from the lab manual of my mentor and collaborator, Dr. Jade Benjamin-Chung. We forked that repo and several of the sections remain intact given the similar coding style that I have adopted after working with Dr. Benjamin-Chung. Original author attributions are shown at the top of

each section. Contributors from the Grembi lab include Dr. Jess Grembi, Pablo Ochoa Andersen, and Nazifa Tabassum.

Feel free to draw from this manual (and please cite it if you do!).

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

Chapter 2

Culture and conduct

by Jess Grembi

2.1 Lab culture

We are committed to a lab culture that fosters creativity, integrity, enthusiasm, inclusivity, and rigor. As an interdisciplinary team, our differences are our strength. We aim to work with each other in a collaborative, supportive, inclusive, and open manner, and our lab space is free from discrimination and harassment.

We aspire to be a lab where everyone feels motivated to share their thoughts and ideas in a respectful and constructive way. Each of us sees things differently and comes to the table with different expertise. Even new lab members who are still learning about our research can offer valuable critical evaluation of our work and are exceptional resources to gain feedback on what is not clear in how we present our research.

We work together to share our knowledge and seek assistance when needed. This lab manual is a resource for sharing such information with each other, in particular how to get your analyses running smoothly. Please let Dr. Grembi know if you have ideas about new content to add!

2.2 Protecting human subjects

All lab members must complete CITI Biomedical Human Subjects Research (IRB) Course Stage 1 - Basic Course training and share their certificate with Jess. She will add team members to relevant Institutional Review Board protocols prior to their start date to ensure they have permission to work with identifiable datasets.

One of the most relevant aspects of protecting human subjects in our work is maintaining confidentiality. For students supporting our data science efforts, in practice this means:

- Be sure to understand and comply with project-specific policies about where data can be saved, particularly if the data include personal identifiers.
- Do not share data with anyone without permission, including to other members of the group, who might not be on the same IRB protocol as you (check with Jess first).

Remember, data that looks like it does not contain identifiers to you might still be classified as data that requires special protection by our IRB or under HIPAA, so always proceed with caution and ask for help if you have any concerns about how to maintain study participant confidentiality.

2.3 Authorship

Team members who meet the ICMJE Definition of authorship will be included as co-authors on scientific manuscripts.

2.4 Responsible use of AI tools

This guidance draws heavily from a version shared by Dr. Kim Meier (Assistant Professor at the University of Houston)

I encourage lab members to think of large language models (LLMs) and other AI tools the way we think about calculators or Wikipedia: they can be incredibly useful, but only if you already have an understanding of what you're doing. They're best used as assistants – not as authorities or substitutes for careful thinking. This guide outlines how I expect AI tools to be used in our lab context. This guide was written with the help of ChatGPT.

2.4.1 Philosophy

AI tools can help us work more efficiently, learn more quickly, and get past sticking points – whether that's debugging a script, brainstorming how to visualize data, or wading through a dense research article. I see them as part of the broader toolkit we use to support our work, like textbooks, search engines, or discussion with labmates. But just like those other tools, the usefulness of AI depends on the thoughtfulness of the person using it. It can help you think, but it can't do the thinking for you. Think of AI tools like a lab whiteboard: a place to sketch ideas, not publish results. It will not take responsibility for errors, misinterpretations, or ethical lapses. That's still on you.

2.4.2 Helpful and encouraged uses

You're welcome to use AI tools to assist with tasks like:

- Googling stuff! Even excluding Gemini, Google search uses AI
- Getting help understanding a research paper or background topic
- Brainstorming or refining ideas (e.g., for a figure, a method, or a statistical approach)
- Getting feedback on a piece of writing you drafted
- Asking for explanations of code behavior or R functions
- Writing or revising your own code, as long as you understand what the code is doing
- Generating visualizations or suggesting ways to summarize data

It's fine to use AI to "talk things out." Just don't stop there – make sure you apply your own judgment and review anything it suggests carefully.

2.4.3 Use with caution: known limitations

AI tools often sound confident even when they're wrong. They can fabricate citations, introduce subtle bugs in code, or suggest statistical approaches that don't fit your design. Some specific risks include:

- Code suggestions might run but be logically incorrect
- Statistical advice might not match your data or assumptions
- Summaries of research papers can miss key details or invent conclusions

While AI tools are improving — for example, ChatGPT now offers a DeepResearch feature designed to reduce citation hallucinations and summarise literature — these are still limited to open-access articles only, and errors still happen. Always verify citations at the source. To use DeepResearch, you have to ask ChatGPT to use it as it won't search carefully by default. For example, you would need your prompt to be something like: "Using deep research, give me a paper that describes X"

If you don't already understand the concept or method, you probably won't be able to tell if the AI got it wrong. That's a sign you should pause and ask a labmate (or me) for help.

2.4.4 Boundaries and responsibilities

- No uploading data: Never paste data, participant responses, or sensitive information into AI tools – even in small samples. That includes data from active studies, even if it looks anonymous. If you're getting help with code, describe the structure of your data instead (e.g. "I have a $64 \times 1000 \times 120$ matrix called `motionData` representing channels x timepoints (in ms) x trials", or "I have a data frame in R where each row represents a participant and the columns include group, age, and threshold"). If you're not

sure how to describe your data without showing it, ask a labmate or bring it to our next meeting.

- **No identifiable content:** Avoid including names or sharing personal details with these tools, including information about yourself - whether you're getting help with writing a reference letter, drafting an email, or talking through a lab-related issue. Even if it seems harmless, AI tools are not private spaces.
- **You're responsible for vetting the output:** If you don't understand the code, math, or language an AI tool gives you, don't just copy and paste it into your project. Run it by someone else, or step back and learn the concept first. That's how you grow as a researcher. AI is here to help you work, not do your work for you.
- **No AI-written research content:** AI should never be used to generate text for a research paper, abstract, or manuscript section. You can use it to help organize your thoughts or improve the clarity of something you wrote - but the core ideas, language, and citations must be your own. Even if you're using AI to help you phrase something better, avoid copy-pasting text that it wrote for you. You'll learn a lot more (and avoid problems) by rewriting in your own words.
- **This includes embedded AI tools:** This policy applies not just to standalone tools like ChatGPT, but also to built-in features like Cursor, GitHub Copilot, Microsoft Editor, or Google Docs suggestions. Use the same judgment no matter where the AI shows up.

2.4.5 Other contexts have other rules

This guide is specific to our lab work. If you're working on coursework, a class presentation, a fellowship application, or a conference submission, you must follow their guidelines about AI use and authorship.

Chapter 3

Communication and coordination

by Jess Grembi

One benefit of the academic environment is its schedule flexibility. This means that lab members may choose to work in the early morning, evening, or weekends. That said, we do not expect lab members to respond outside of business hours (unless there are special circumstances).

3.1 Slack

- Use Slack for scheduling, coding related questions, quick check ins, etc. If your Slack message exceeds 200 words, it might be time to use email.
- Note that our current Slack subscription only allows us to access messages sent within the previous 90 days. So, ensure that critical information is saved elsewhere.
- Use channels instead of direct messages unless you need to discuss something private.
- Please make an effort to respond to messages that message you (e.g., @jess) as quickly as possible and always within 24 hours.
- If you are unusually busy (e.g., taking MCAT/GRE, taking many exams) or on vacation please alert the team in advance so we can expect you not to respond at all / as quickly as usual and also set your status in Slack (e.g., it could say “On vacation”) so we know not to expect to see you online.
- Please thread messages in Slack as much as possible.

3.2 Email

- Use email for longer messages (>200 words) or messages that merit preservation.
- Generally, strive to respond within 24 hours. As noted above, if you are unusually busy or on vacation please alert the team in advance so we can expect you not to respond at all / as quickly as usual.

3.3 Trello

- Jess will add new cards within our shared Trello board that outline your tasks.
- The higher a card is within your list, the higher priority it is.
- Generally, strive to complete the tasks in your card by the date listed.
- Use checklists to break down a task into smaller chunks. Sometimes Jess will write this for you, but you can also add this yourself.
- Please move your card to the “Completed” list when it is done. Cards with ‘#TJ’ in the title indicate that you should “Tell Jess” when the task has been completed. This hashtag is an indicator that someone else might be waiting on you to finish this task in order to proceed with other work, so it’s another way to ensure critical tasks are communicated within the team.

3.4 Sharepoint

- We mostly use Sharepoint to create shared documents with longer descriptions of tasks. These documents are linked to in Trello. We often share these with the whole team since tasks are overlapping, and even if a task is assigned to one person, others may have valuable insights.

3.5 Meetings

- Our meetings start on the hour.
- If you are going to be late, please send a message in our Slack channel.
- If you are regularly not able to come on the hour, notify the team and we might choose to modify the agenda order or the start time.

Chapter 4

Lab Safety

by Jess Grembi

Safety, health, and environmental stewardship are priorities at Penn State. In particular, our lab handles samples that could potentially contain BSL2 organisms - these are pathogens that can cause diarrheal illness that is self-limiting (not life threatening). We do not want to get ourselves or others sick so we adhere to strict protocols to reduce exposure when working in the lab.

The Environmental Health and Safety (EHS) team at PSU is a resource to help labs ensure we are in compliance with Federal and State laws, University directives, and are doing our best to support safety, health, and environmental stewardship. If you have any questions about these topics, feel free to reach out to Dr. Grembi or the lab Safety Officer. If we can't answer your question, we can reach out to EHS for their assistance.

The lab Safety Officer is Pablo Ochoa-Andersen

4.1 Key points regarding safety:

- Appropriate lab attire includes closed toe shoes and long pants.
- Do not keep food or drink in the lab area.
- Wash your hands before and after conducting lab work.
- Gloves are not worn when opening doors or touching objects that may spread contamination to others.
- All gloves are permeable and will eventually allow chemicals and biological agents to come into contact with your skin. Thus, double glove if you plan to work with any harsh chemicals and replace gloves as soon as feasible after they become contaminated.

- When using Biosafety cabinets ensure that you are not restricting airflow (with your body or with equipment placed into the cabinet).

4.2 Required Institutional Lab Safety Training

There are several required trainings before anyone can work in the lab. Please ensure you have completed all trainings *and provided certificates to the Lab Safety Officer* before performing any work in the lab.

4.2.1 The following trainings can be accessed from the EHS training website

- Hazard Communication (HazCom) Training
- Laboratory and Research Safety Training (initial and then annual refresher)
- Biosafety cabinet training
- Regulated Medical Waste
- Spill Prevention and Response
- Compressed gas training (if using the Coy)
- Autoclave Safety (if using the autoclave)

4.2.2 Our lab is a Biosafety Level 2 (BSL2) lab, which requires the following trainings that can be accessed from the CITI website

- Yearly Biosafety training
- Bloodborne Pathogens - Research Personnel

4.2.3 Since we work with Bloodborne Pathogens, everyone working in the lab needs to be offered updated Hepatitis B vaccination

- The Bloodborne Pathogen Immunization Determination form can be found [here](#)
- This needs to be sent to Occupational Medicine, a normal course is two vaccinations one month apart

4.3 Lab Safety Tour and site-specific hazard identification

Prior to conducting any work in the lab, new members will be given a lab safety tour by the Safety Officer where you will be shown the major hazards in the lab, and introduced to our lab-specific safety protocols. The safety tour will conclude with a review the Unit Specific Plan, and you will be asked to sign

that you understand the hazards present in the lab and the protocols in place to minimize risk.

4.4 Lab Safety Officer Responsibilities

- Serve as the Chemical Safety Officer (CHO)
- Weekly Hazardous waste accumulation point check
- Weekly eyewash station inspection and testing
- Monthly safety inspections

Chapter 5

Reproducibility

by Jade Benjamin-Chung

Our lab adopts the following practices to maximize the reproducibility of our work.

1. Design studies with appropriate methodology and adherence to best practices in epidemiology and biostatistics
2. Register study protocols
3. Write and register pre-analysis plans
4. Create reproducible workflows
5. Process and analyze data with internal replication and masking
6. Use reporting checklists with manuscripts
7. Publish preprints
8. Publish data (when possible) and replication scripts

5.1 What is the reproducibility crisis?

In the past decade, an increasing number of studies have found that published study findings could not be reproduced. Researchers found that it was not possible to reproduce estimates from published studies: 1) with the same data and same or similar code and 2) with newly collected data using the same (or similar) study design. These “failures” of reproducibility were frequent enough and broad enough in scope, occurring across a range of disciplines (epidemiology, psychology, economics, and others) to be deeply troubling. Program and policy decisions based on erroneous research findings could lead to wasted resources, and at worst, could harm intended beneficiaries. This crisis has motivated new practices in reproducibility, transparency, and openness. Our lab is committed to adopting these best practices, and much of the remainder of the lab manual focuses on how to do so.

Recommended readings on the “reproducibility crisis”:

- Nuzzo R. How scientists fool themselves – and how they can stop. 2015. <https://www.nature.com/articles/526182a>
- Stoddart C. Is there a reproducibility crisis in science? 2016. <https://www.nature.com/articles/d41586-019-00067-3>
- Munafo MR, et al. A manifesto for reproducible science. *Nature Human Behavior* 2017 <http://dx.doi.org/10.1038/s41562-016-0021>

5.2 Study design

Appropriate study design is beyond the scope of this lab manual and is something trainees develop through their coursework and mentoring.

5.3 Register study protocols

We register all randomized trials on clinicaltrials.gov, and in some cases register observational studies as well.

5.4 Write and register pre-analysis plans

We write pre-analysis plans for most original research projects that are not exploratory in nature, although in some cases, we write pre-analysis plans for exploratory studies as well. The format and content of pre-analysis plans can vary from project to project. Here is an example of one: <https://osf.io/tgbxr/>. Generally, these include:

1. Brief background on the study (a condensed version of the introduction section of the paper)
2. Hypotheses / objectives
3. Study design
4. Description of data
5. Definition of outcomes
6. Definition of interventions / exposures
7. Definition of covariates
8. Statistical power calculation
9. Statistical analysis:
 - Type of model
 - Covariate selection / screening
 - Standard error estimation method
 - Missing data analysis
 - Assessment of effect modification / subgroup analyses
 - Sensitivity analyses

- Negative control analyses

5.5 Create reproducible workflows

Reproducible workflows allow a user to reproduce study estimates and ideally figures and tables with a “single click”. In practice, this typically means running a single bash script that sources all replication scripts in a repository. These replication scripts complete data processing, data analysis, and figure/table generation. The following chapters provide detailed guidance on this topic:

- Chapter 5: Code repositories
- Chapter 6: Coding practices
- Chapter 7: Coding style
- Chapter 8: Code publication
- Chapter 9: Working with big data
- Chapter 10: Github
- Chapter 11: Unix

5.6 Process and analyze data with internal replication and masking

See my colleague Jade Benjamin-Chung’s video on this topic: <https://www.youtube.com/watch?v=WoYkY9MkbRE>

5.7 Use reporting checklists with manuscripts

Using reporting checklists helps ensure that peer-reviewed articles contain the information needed for readers to assess the validity of your work and/or attempt to reproduce it. A collection of reporting checklists is available here: <https://www.equator-network.org/about-us/what-is-a-reporting-guideline/>

5.8 Publish preprints

A preprint is a scientific manuscript that has not been peer reviewed. Preprint servers create digital object identifiers (DOIs) and can be cited in other articles and in grant applications. Because the peer review process can take many months, publishing preprints prior to or during peer review enables other scientists to immediately learn from and build on your work. Importantly, NIH allows applicants to include preprint citations in their biosketches. In most cases, we publish preprints on medRxiv.

5.9 Publish data (when possible) and replication scripts

Publishing data and replication scripts allows other scientists to reproduce your work and to build upon it. We typically publish data on Open Science Framework, share links to Github repositories, and archive code on Zenodo.

Chapter 6

Code repositories

By Jess Grembi, Kunal Mishra, Jade Benjamin-Chung, and Stephanie Djajadi

Each study has a code repository that typically holds R code, shell scripts with Unix code, and research outputs (results .RDS files, tables, figures). Repositories may also include datasets if they do not contain PHI/PII. Our lab uses GitHub to organize and share our repositories both internally and publicly when we are ready to publish our results. This chapter outlines how to organize files within the repository and how to interface with GitHub. Adhering to a standard format makes it easier for us to efficiently collaborate across projects.

6.1 Project Structure

We recommend the following directory structure:

```
0-run-project.sh
0-config.R
0 - Utils/
    0-base-functions.R
1 - Data-Management/
    0-prep-data.sh
    1-prep-cdph-fluseas.R
    2a-prep-absentee.R
    2b-prep-absentee-weighted.R
    3a-prep-absentee-adj.R
    3b-prep-absentee-adj-weighted.R
2 - Analysis/
    0-run-analysis.sh
    1 - Absentee-Mean/
        1-absentee-mean-primary.R
        2-absentee-mean-negative-control.R
```

```

    3-absentee-mean-CDC.R
    4-absentee-mean-peakwk.R
    5-absentee-mean-cdph2.R
    6-absentee-mean-cdph3.R
  2 - Absentee-Positivity-Check/
    3 - Absentee-P1/
    4 - Absentee-P2/
3 - Figures/
  0-run-figures.sh
  ...
4 - Tables/
  0-run-tables.sh
  ...
5 - Results/
  1 - Absentee-Mean/
    1-absentee-mean-primary.RDS
    2-absentee-mean-negative-control.RDS
    3-absentee-mean-CDC.RDS
    4-absentee-mean-peakwk.RDS
    5-absentee-mean-cdph2.RDS
    6-absentee-mean-cdph3.RDS
  ...
.gitignore
.Rproj

```

For brevity, not every directory is “expanded”, but we can glean some important takeaways from what we *do* see.

Examples from previous projects can be found here: *[Zi yi, add the links to the github examples here]*

6.2 .Rproj files

An “R Project” can be created within RStudio by going to **File >> New Project**. Depending on where you are with your research, choose the most appropriate option. This will save preferences, working directories, and even the results of running code/data (though I’d recommend starting from scratch each time you open your project, in general). Then, ensure that whenever you are working on that specific research project, you open your created project to enable the full utility of .Rproj files. This also automatically sets the directory to the top level of the project.

6.3 Configuration ('config') File

This is the single most important file for your project. It will be responsible for a variety of common tasks, declare global variables, load functions, declare paths, and more. *Every other file in the project* will begin with `source("0-config")`, and its role is to reduce redundancy and create an abstraction layer that allows you to make changes in one place (`0-config.R`) rather than 5 different files. To this end, paths which will be reference in multiple scripts (i.e. a `merged_data_path`) can be declared in `0-config.R` and simply referred to by its variable name in scripts. If you ever want to change things, rename them, or even switch from a downsample to the full data, all you would then need to do is modify the path in one place and the change will automatically update throughout your project. See the example config file for more details. The paths defined in the `0-config.R` file assume that users have opened the `.Rproj` file, which sets the directory to the top level of the project.

6.4 Order Files and Directories

This makes the jumble of alphabetized filenames much more coherent and places similar code and files next to one another. This also helps us understand how data flows from start to finish and allows us to easily map a script to its output (i.e. `2 - Analysis/1 - Absentee-Mean/1-absentee-mean-primary.R => 5 - Results/1 - Absentee-Mean/1-absentee-mean-primary.RDS`). If you take nothing else away from this guide, this is the single most helpful suggestion to make your workflow more coherent. Often the particular order of files will be in flux until an analysis is close to completion. At that time it is important to review file order and naming and reproduce everything prior to drafting a manuscript.

6.5 Using Bash scripts to ensure reproducibility

Bash scripts are useful components of a reproducible workflow. At many of the directory levels (i.e. in `3 - Analysis`), there is a bash script that runs each of the analysis scripts. This is exceptionally useful when data “upstream” changes – you simply run the bash script. See the Unix Chapter for further details.

After running bash scripts, `.Rout` log files will be generated for each script that has been executed. It is important to check these files. Scripts may appear to have run correctly in the terminal, but checking the log files is the only way to ensure that everything has run completely.

Chapter 7

Coding practices

by Kunal Mishra, Jade Benjamin-Chung, Stephanie Djajadi, and Iris Tong

7.1 Organizing scripts

Just as your data “flows” through your project, data should flow naturally through a script. Very generally, you want to:

1. describe the work completed in the script in a comment header
2. source your configuration file (`0-config.R`)
3. load all your data
4. do all your analysis/computation
5. save your data.

Each of these sections should be “chunked together” using comments. See this file for a good example of how to cleanly organize a file in a way that follows this “flow” and functionally separate pieces of code that are doing different things.

7.2 Documenting your code

7.2.1 File headers

Every file in a project should have a header that allows it to be interpreted on its own. It should include the name of the project and a short description for what this file (among the many in your project) does specifically. You may optionally wish to include the inputs and outputs of the script as well, though the next section makes this significantly less necessary.

```
#####  
# @Organization - Example Organization  
# @Project - Example Project
```

```
# @Description - This file is responsible for [...]
#####
```

7.2.2 Sections and subsections

Rstudio (v1.4 or more recent) supports the use of Sections and Subsections. You can easily navigate through longer scripts using the navigation pane in RStudio, as shown on the right below.

```
# Section -----

## Subsection -----

### Sub-subsection -----
```

7.2.3 Code folding

Consider using RStudio’s code folding feature to collapse and expand different sections of your code. Any comment line with at least four trailing dashes (-), equal signs (=), or pound signs (#) automatically creates a code section. See the above link for more info.

7.2.4 Comments in the body of your script

Commenting your code is an important part of reproducibility and helps document your code for the future. When things change or break, you’ll be thankful for comments. There’s no need to comment excessively or unnecessarily, but a comment describing what a large or complex chunk of code does is always helpful. See this file for an example of how to comment your code and notice that comments are always in the form of:

```
# This is a comment -- first letter is capitalized and spaced
away from the pound sign
```

7.2.5 Function documentation

Every function you write must include a header to document its purpose, inputs, and outputs. For any reproducible workflows, they are essential, because R is dynamically typed. This means, you can pass a **string** into an argument that is meant to be a **data.table**, or a **list** into an argument meant for a **tibble**. It is the responsibility of a function’s author to document what each argument is meant to do and its basic type. This is an example for documenting a function (inspired by JavaDocs and R’s Plumber API docs):

```
#####
#####
# Documentation: calc_fluseas_mean
# Usage: calc_fluseas_mean(data, yname)
```

```

# Description: Make a dataframe with rows for flu season and site
# and the number of patients with an outcome, the total patients,
# and the percent of patients with the outcome

# Args/Options:
# data: a data frame with variables flu_season, site, studyID, and yname
# yname: a string for the outcome name
# silent: a boolean specifying whether the function shouldn't output anything to the console (DEF

# Returns: the dataframe as described above
# Output: prints the data frame described above if silent is not True

calc_fluseas_mean = function(data, yname, silent = TRUE) {
  ### function code here
}

```

The header tells you what the function does, its various inputs, and how you might go about using the function to do what you want. Also notice that all optional arguments (i.e. ones with pre-specified defaults) follow arguments that require user input.

- **Note:** As someone trying to call a function, it is possible to access a function's documentation (and internal code) by **CMD-Left-Clicking** the function's name in RStudio
- **Note:** Depending on how important your function is, the complexity of your function code, and the complexity of different types of data in your project, you can also add "type-checking" to your function with the `assertthat::assert_that()` function. You can, for example, `assert_that(is.data.frame(statistical_input))`, which will ensure that collaborators or reviewers of your project attempting to use your function are using it in the way that it is intended by calling it with (at the minimum) the correct type of arguments. You can extend this to ensure that certain assumptions regarding the inputs are fulfilled as well (i.e. that `time_column`, `location_column`, `value_column`, and `population_column` all exist within the `statistical_input` tibble).

7.3 Object naming

Generally we recommend using nouns for objects and verbs for functions. This is because functions are performing actions, while objects are not.

Try to make your variable names both more expressive and more explicit. Being a bit more verbose is useful and easy in the age of autocompletion! For example, instead of naming a variable `vaxcov_1718`, try naming it `vaccination_coverage_2017_18`. Similarly, `flu_res` could be named

`absentee_flu_residuals`, making your code more readable and explicit.

- For more help, check out *Be Expressive: How to Give Your Variables Better Names*

We recommend you use **Snake_Case**.

- Base R allows `.` in variable names and functions (such as `read.csv()`), but this goes against best practices for variable naming in many other coding languages. For consistency's sake, **snake_case** has been adopted across languages, and modern packages and functions typically use it (i.e. `readr::read_csv()`). As a very general rule of thumb, if a package you're using doesn't use **snake_case**, there may be an updated version or more modern package that *does*, bringing with it the variety of performance improvements and bug fixes inherent in more mature and modern software.
- **Note:** you may also see `camelCase` throughout the R code you come across. This is *okay* but not ideal – try to stay consistent across all your code with **snake_case**.
- **Note:** again, its also worth noting there's nothing inherently wrong with using `.` in variable names, just that it goes against style best practices that are cropping up in data science, so its worth getting rid of these bad habits now.

7.4 Function calls

In a function call, use “named arguments” and put each argument on a separate line to make your code more readable.

Here's an example of what not to do when calling the function a function `calc_fluseas_mean` (defined above):

```
mean_Y = calc_fluseas_mean(flu_data, "maari_yn", FALSE)
```

And here it is again using the best practices we've outlined:

```
mean_Y = calc_fluseas_mean(
  data = flu_data,
  yname = "maari_yn",
  silent = FALSE
)
```

7.5 The here package

The **here** package is one great R package that helps multiple collaborators deal with the mess that is working directories within an R project structure. Let's

say we have an R project at the path `/home/oski/Some-R-Project`. My collaborator might clone the repository and work with it at some other path, such as `/home/bear/R-Code/Some-R-Project`. Dealing with working directories and paths explicitly can be a very large pain, and as you might imagine, setting up a Config with paths requires those paths to flexibly work for all contributors to a project. This is where the `here` package comes in and this a great vignette describing it.

7.6 Reading/Saving Data

7.6.1 .RDS vs .RData Files

One of the most common ways to load and save data in Base R is with the `load()` and `save()` functions to serialize multiple objects in a single `.RData` file. The biggest problems with this practice include an inability to control the names of things getting loaded in, the inherent confusion this creates in understanding older code, and the inability to load individual elements of a saved file. For this, we recommend using the RDS format to save R objects.

- **Note:** if you have many related R objects you would have otherwise saved all together using the `save` function, the functional equivalent with RDS would be to create a (named) list containing each of these objects, and saving it.

7.6.2 CSVs

Once again, the `readr` package as part of the Tidvyerse is great, with a much faster `read_csv()` than Base R's `read.csv()`. For massive CSVs (> 5 GB), you'll find `data.table::fread()` to be the fastest CSV reader in any data science language out there. For writing CSVs, `readr::write_csv()` and `data.table::fwrite()` outclass Base R's `write.csv()` by a significant margin as well.

7.7 Integrating Box and Dropbox

Box and Dropbox are cloud-based file sharing systems that are useful when dealing with large files. When our scripts generate large output files, the files can slow down the workflow if they are pushed to GitHub. This makes collaboration difficult when not everyone has a copy of the file, unless we decide to duplicate files and share them manually. The files might also take up a lot of local storage. Box and Dropbox help us avoid these issues by automatically storing the files, reading data, and writing data back to the cloud.

Box and Dropbox are separate platforms, but we can use either one to store and share files. To use them, we can install the packages that have been created to integrate Box and Dropbox into R. The set-up instructions are detailed below.

Make sure to authenticate before reading and writing from either Box or Dropbox. The authentication commands should go in the configuration file; it only needs to be done once. This will prompt you to give your login credentials for Box and Dropbox and will allow your application to access your shared folders.

7.7.1 Box

Follow the instructions in this section to use the `boxr` package. Note that there are a few setup steps that need to be done on the box website before you can use the `boxr` package, explained here in the section “Creating an Interactive App.” This gets the authentication keys that must be put in box. Once that is done, add the authentication keys to your code in the configuration file, with `box_auth(client_id = "<your_client_id>", client_secret = "<your_client_secret_id>")`. It is also important to set the default working directory so that the code can reference the correct folder in box: `box_setwd(<folder_id>)`. The folder ID is the sequence of digits at the end of the URL.

Further details can be found [here](#).

7.7.2 Dropbox

Follow the instructions at this [link](#) to use the `rdrop2` package. Similar to the `boxr` package, you must authenticate before reading and writing from Dropbox, which can be done by adding `drop_auth()` to the configuration file.

Saving the authentication token is not required, although it may be useful if you plan on using Dropbox frequently. To do so, save the token with the following commands. Tokens are valid until they are manually revoked.

```
# first time only
# save the output of drop_auth to an RDS file
token <- drop_auth()
# this token only has to be generated once, it is valid until revoked
saveRDS(token, "/path/to/tokenfile.RDS")

# all future usages
# to use a stored token, provide the rdtoken argument
drop_auth(rdtoken = "/path/to/tokenfile.RDS")
```

7.8 Tidyverse

Throughout this document there have been references to the Tidyverse, but this section is to explicitly show you how to transform your Base R tendencies to Tidyverse (or `Data.Table`, Tidyverse’s performance-optimized competitor). For most of our work that does not utilize very large datasets, we recommend that you code in Tidyverse rather than Base R. Tidyverse is quickly becoming the

gold standard in R data analysis and modern data science packages and code should use Tidyverse style and packages unless there's a significant reason not to (i.e. big data pipelines that would benefit from Data.Table's performance optimizations).

The package author has published a great textbook on R for Data Science, which leans heavily on many Tidyverse packages and may be worth checking out.

The following list is not exhaustive, but is a compact overview to begin to translate Base R into something better:

Base R	Better Style, Performance, and Utility
<code>read.csv()</code>	<code>readr::read_csv()</code> or <code>data.table::fread()</code>
<code>write.csv()</code>	<code>readr::write_csv()</code> or <code>data.table::fwrite()</code>
<code>readRDS</code> <code>saveRDS()</code>	<code>readr::read_rds()</code> <code>readr::write_rds()</code>
<code>data.frame()</code>	<code>tibble::tibble()</code> or <code>data.table::data.table()</code>
<code>rbind()</code> <code>cbind()</code> <code>df\$some_column</code> <code>df\$some_column = ...</code>	<code>dplyr::bind_rows()</code> <code>dplyr::bind_cols()</code> <code>df %>% dplyr::pull(some_column)</code> <code>df %>% dplyr::mutate(some_column = ...)</code>
<code>df[get_rows_condition,]</code>	<code>df %>% dplyr::filter(get_rows_condition)</code>
<code>df[,c(col1, col2)]</code>	<code>df %>% dplyr::select(col1, col2)</code>
<code>merge(df1, df2, by = ..., all.x = ..., all.y = ...)</code>	<code>df1 %>% dplyr::left_join(df2, by = ...) or dplyr::full_join or dplyr::inner_join or dplyr::right_join</code>
<code>str()</code> <code>grep(pattern, x)</code> <code>gsub(pattern, replacement, x)</code>	<code>dplyr::glimpse()</code> <code>stringr::str_which(string, pattern)</code> <code>stringr::str_replace(string, pattern, replacement)</code>
<code>ifelse(test_expression, yes, no)</code>	<code>if_else(condition, true, false)</code>

Base R	Better Style, Performance, and Utility
Nested: <code>ifelse(test_expression1, yes1, ifelse(test_expression2, yes2, ifelse(test_expression3, yes3, no)))</code>	<code>case_when(test_expression1 ~ yes1, test_expression2 ~ yes2, test_expression3 ~ yes3, TRUE ~ no)</code>
<code>proc.time()</code>	<code>tictoc::tic()</code> and <code>tictoc::toc()</code>
<code>stopifnot()</code>	<code>assertthat::assert_that()</code> or <code>assertthat::see_if()</code> or <code>assertthat::validate_that()</code>

For a more extensive set of syntactical translations to Tidyverse, you can check out this document.

Working with Tidyverse within functions can be somewhat of a pain due to non-standard evaluation (NSE) semantics. If you're an avid function writer, we'd recommend checking out the following resources:

- Tidy Eval in 5 Minutes (video)
- Tidy Evaluation (e-book)
- Data Frame Columns as Arguments to Dplyr Functions (blog)
- Standard Evaluation for `*_join` (stackoverflow)
- Programming with dplyr (package vignette)

7.9 Coding with R and Python

If you're using both R and Python, you may wish to check out the Feather package for exchanging data between the two languages extremely quickly.

7.10 Repeating analyses with different variations

In many cases, we will need to apply our modeling on different combinations of interests (outcomes, exposures, etc.). We can certainly use a `for` loop to repeat the execution of a wrapper function, but generally, `for` loops request high memory usage and produce the results in long computation time.

Fortunately, R has some functions which implement looping in a compact form to help repeating your analyses with different variations (subgroups, outcomes, covariate sets, etc.) with better performances.

7.10.1 `lapply()` and `sapply()`

`lapply()` is a function in the base R package that applies a function to each element of a list and returns a list. It's typically faster than `for`. Here is a

simple generic example:

```
result <- lapply(X = mylist, FUN = func)
```

There is another very similar function called `sapply()`. It also takes a list as its input, but if the output of the `func` is of the same length for each element in the input list, then `sapply()` will simplify the output to the simplest data structure possible, which will usually be a vector.

7.10.2 `mapply()` and `pmap()`

Sometimes, we'd like to employ a wrapper function that takes arguments from multiple different lists/vectors. Then, we can consider using `mapply()` from the base R package or `pmap()` from the `purrr` package.

Please see the simple specific example below where the two input lists are of the same length and we are doing a pairwise calculation:

```
mylist1 = list(0:3)
mylist2 = list(6:9)
mylists = list(mylist1, mylist2)

square_sum <- function(x, y) {
  x^2 + y^2
}

#Use `mapply()``
result1 <- mapply(FUN = square_sum, mylist1, mylist2)

#Use `pmap()``
library(purrr)
result2 <- pmap(.l = mylists, .f = square_sum)

#unlist(as.list(result1)) = result2 = [36 50 68 90]
```

There are two major differences between `mapply()` and `pmap()`. The first difference is that `mapply()` takes separate lists as its input arguments, while `pmap()` takes a list of list. Secondly, the output of `mapply()` will be in the form of a matrix or an array, but `pmap()` produces a list directly.

However, when the input lists are of different lengths AND/OR the wrapper function doesn't take arguments in pairs, `mapply()` and `pmap()` may not give the preferable results.

Both `mapply()` and `pmap()` will recycle shorter input lists to match the length of the longest input list. Assume that now `mylist2 = list(6:12)`. Then, `pmap(mylists, square_sum)` will generate `[36 50 68 90 100 122 148]` where elements 0, 1, and 2 are recycled to match 10, 11, and 12. And it will

return an error message that “longer object length is not a multiple of shorter object length.”

Thus, unless the recycling pattern described above is desirable feature for a certain experiment design, **when the input lists are of different lengths, the best practice is probably to use `lapply()` and then combine the results.**

Here is an example where we’d like to find the `square_sum` for every element combination of `mylist1` and `mylist2`.

```
mylist1 <- list(0:3)
mylist2 <- list(6:12)

square_sum <- function(x, y) {
  x^2 + y^2
}

results <- list()

for (i in seq_along(mylist1[[1]])) {
  result <- lapply(X = mylist2, FUN = function(y) square_sum(mylist1[[1]][i], y))
  results[[i]] <- result
}
```

This example doesn’t work in the way that 0 is paired to 6, 1 is paired to 7, and so on. Instead, every element in `mylist1` will be paired with every element in `mylist2`. Thus, the “unlisted” results from the example will have $4 * 7 = 28$ elements.

We can use `flatten()` or `unlist()` functions to decrease the depths of our results. If the results are data frames, then we will need to use `bind_rows()` to combine them.

7.10.3 Parallel processing with `parallel` and `future` packages

One big drawback of `lapply()` is its long computation time, especially when the list length is long. Fortunately, computers nowadays must have multiple cores which makes parallel processing possible to help make computation much faster.

Assume you have a list called `mylist` of length 1000, and `lapply(X = mylist, FUN = func)` applies the function to each of the 1000 elements one by one in T seconds. If we could execute the `func` in n processors simultaneously, then ideally, we would shrink the computation time to T/n seconds.

In practice, using functions under the `parallel` and the `future` packages, we can split `mylist` into smaller chunks and apply the function to each element of

the several chunks in parallel in different cores to significantly reduce the run time.

7.10.3.1 parLapply()

Below is a generic example of `parLapply()`:

```
library(parallel)

# Set how many processors will be used to process the list and make cluster
n_cores <- 4
cl <- makeCluster(n_cores)

#Use parLapply() to apply func to each element in mylist
result <- parLapply(cl = cl, x = mylist, FUN = func)

#Stop the parallel processing
stopCluster(cl)
```

Let's still assume `mylist` is of length 1000. The `parLapply` above splits `mylist` into 4 sub-lists each of length 250 and applies the function to the elements of each sub-list in parallel. To be more specific, first apply the function to element 1, 251, 501, 751; second apply to element 2, 252, 502, 752; so on and so forth. As such, the computation time will be greatly reduced.

You can use `parallel::detectCores()` to test how many cores your machine has and to help decide what to put for `n_cores`. It would be a good idea to leave at least one core free for the operating system to use.

We will always start `parLapply()` with `makeCluster()`. **`stopCluster()` is not fully necessary but follows the best practices.** If not stopped, the processing will continue in the back end and consuming the computation capacity for other software in your machine. But keep in mind that stopping the cluster is similar quitting R, meaning that you will need to re-load the packages needed when you need to do parallel processing use `parLapply()` again.

7.10.3.2 future.lapply()

Below is a generic example of `future.lapply()`:

```
library(future)
library(future.apply)

# First, plan how the future_lapply() will be resolved
future::plan(
  multisession, workers = future::availableCores() - 1
)

# Use future_lapply() to apply func to each element in mylist
```

```
future_lapply(x = mylist, FUN = func)
```

Here, `future::availableCores()` checks how many cores your machine has. Similar to `parLapply()` showed above, `future_lapply()` parallelizes the computation of `lapply()` by executing the function `func` simultaneously on different sub-lists of `mylist`.

7.11 Reviewing Code

Before publishing new changes, it is important to ensure that the code has been tested and well-documented. GitHub makes it possible to document all of these changes in a pull request. Pull requests can be used to describe changes in a branch that are ready to be merged with the base branch (more information in the GitHub section). Github allows users to create a pull request template in a repository to standardize and customize the information in a pull request. When you add a pull request template to your repository, everyone will automatically see the template's contents in the pull request body.

7.11.1 Creating a Pull Request Template

Follow the instructions below to add a pull request template to a repository. More details can be found at this [GitHub link](#).

1. On GitHub, navigate to the main page of the repository.
2. Above the file list, click **Create new file**.
3. Name the file `pull_request_template.md`. GitHub will not recognize this as the template if it is named anything else. The file must be on the **master** branch.
 1. To store the file in a hidden directory instead of the main directory, name the file `.github/pull_request_template.md`.
4. In the body of the new file, add your pull request template. This could include:
 - A summary of the changes proposed in the pull request
 - How the change has been tested
 - @mentions of the person or team responsible for reviewing proposed changes

Here is an example pull request template.

```
# Description
```

```
## Summary of change
```

```
Please include a summary of the change, including any new functions added and example u
```

```
## Link to Spec
```

Please include a link to the Trello card or Google document with details of the task.

Who should review the pull request?

@ ...

Chapter 8

Coding style

by Kunal Mishra, Jade Benjamin-Chung, and Stephanie Djajadi

8.1 Comments

1. **File Headers** - Every file in a project should have a header that allows it to be interpreted on its own. It should include the name of the project and a short description for what this file (among the many in your project) does specifically. You may optionally wish to include the inputs and outputs of the script as well, though the next section makes this significantly less necessary.

```
#####  
# @Organization - Example Organization  
# @Project - Example Project  
# @Description - This file is responsible for [...]  
#####
```

2. **File Structure** - Just as your data “flows” through your project, data should flow naturally through a script. Very generally, you want to 1) source your config => 2) load all your data => 3) do all your analysis/computation => save your data. Each of these sections should be “chunked together” using comments. See this file for a good example of how to cleanly organize a file in a way that follows this “flow” and functionally separate pieces of code that are doing different things.

- **Note:** If your computer isn’t able to handle this workflow due to RAM or requirements, modifying the ordering of your code to accommodate it won’t be ultimately helpful and your code will be fragile, not to mention less readable and messy. You need to look into high-performance computing (HPC) resources in this case.

3. **Single-Line Comments** - Commenting your code is an important part of reproducibility and helps document your code for the future. When things change or break, you'll be thankful for comments. There's no need to comment excessively or unnecessarily, but a comment describing what a large or complex chunk of code does is always helpful. See this file for an example of how to comment your code and notice that comments are always in the form of:

```
# This is a comment -- first letter is capitalized and spaced
away from the pound sign
```

4. **Multi-Line Comments** - Occasionally, multi-line comments are necessary. Don't add line breaks manually to a single-line comment for the purpose of making it "fit" on the screen. Instead, in RStudio > Tools > Global Options > Code > "Soft-wrap R source files" to have lines wrap around. Format your multi-line comments like the file header from above.

8.2 Line breaks

- For `ggplot` calls and `dplyr` pipelines, do not crowd single lines. Here are some nontrivial examples of "beautiful" pipelines, where beauty is defined by coherence:

```
# Example 1
school_names = list(
  OUSD_school_names = absentee_all %>%
    filter(dist.n == 1) %>%
    pull(school) %>%
    unique %>%
    sort,

  WCCSD_school_names = absentee_all %>%
    filter(dist.n == 0) %>%
    pull(school) %>%
    unique %>%
    sort
)

# Example 2
absentee_all = fread(file = raw_data_path) %>%
  mutate(program = case_when(schoolyr %in% pre_program_schoolyrs ~ 0,
                             schoolyr %in% program_schoolyrs ~ 1)) %>%
  mutate(period = case_when(schoolyr %in% pre_program_schoolyrs ~ 0,
                             schoolyr %in% LAIV_schoolyrs ~ 1,
                             schoolyr %in% IIV_schoolyrs ~ 2)) %>%
  filter(schoolyr != "2017-18")
```

And of a complex `ggplot` call:

```
# Example 3
ggplot(data=data,
       mapping=aes_string(x="year", y="rd", group=group)) +

  geom_point(mapping=aes_string(col=group, shape=group),
            position=position_dodge(width=0.2),
            size=2.5) +

  geom_errorbar(mapping=aes_string(ymin="lb", ymax="ub", col=group),
              position=position_dodge(width=0.2),
              width=0.2) +

  geom_point(position=position_dodge(width=0.2),
            size=2.5) +

  geom_errorbar(mapping=aes(ymin=lb, ymax=ub),
              position=position_dodge(width=0.2),
              width=0.1) +

  scale_y_continuous(limits=limits,
                    breaks=breaks,
                    labels=breaks) +

  scale_color_manual(std_legend_title, values=cols, labels=legend_label) +
  scale_shape_manual(std_legend_title, values=shapes, labels=legend_label) +
  geom_hline(yintercept=0, linetype="dashed") +
  xlab("Program year") +
  ylab(yaxis_lab) +
  theme_complete_bw() +
  theme(strip.text.x = element_text(size = 14),
        axis.text.x = element_text(size = 12)) +
  ggtitle(title)
```

Imagine (or perhaps mournfully recall) the mess that can occur when you don't strictly style a complicated `ggplot` call. Trying to fix bugs and ensure your code is working can be a nightmare. Now imagine trying to do it with the same code 6 months after you've written it. Invest the time now and reap the rewards as the code practically explains itself, line by line.

8.3 Automated Tools for Style and Project Workflow

8.3.1 Styling

1. **Code Autoformatting** - RStudio includes a fantastic built-in utility (keyboard shortcut: **CMD-Shift-A**) for autoformatting highlighted chunks of code to fit many of the best practices listed here. It generally makes code more readable and fixes a lot of the small things you may not feel like fixing yourself. Try it out as a “first pass” on some code of yours that *doesn't* follow many of these best practices!
2. **Assignment Aligner** - A cool R package allows you to very powerfully format large chunks of assignment code to be much cleaner and much more readable. Follow the linked instructions and create a keyboard shortcut of your choosing (recommendation: **CMD-Shift-Z**). Here is an example of how assignment aligning can dramatically improve code readability:

Before

```

OUSD_not_found_aliases = list(
  "Brookfield Village Elementary" = str_subset(string = OUSD_school_shapes$schnam, pat
  "Carl Munck Elementary" = str_subset(string = OUSD_school_shapes$schnam, pattern = "
  "Community United Elementary School" = str_subset(string = OUSD_school_shapes$schnam
  "East Oakland PRIDE Elementary" = str_subset(string = OUSD_school_shapes$schnam, pat
  "EnCompass Academy" = str_subset(string = OUSD_school_shapes$schnam, pattern = "EnCom
  "Global Family School" = str_subset(string = OUSD_school_shapes$schnam, pattern = "G
  "International Community School" = str_subset(string = OUSD_school_shapes$schnam, pat
  "Madison Park Lower Campus" = "Madison Park Academy TK-5",
  "Manzanita Community School" = str_subset(string = OUSD_school_shapes$schnam, patter
  "Martin Luther King Jr Elementary" = str_subset(string = OUSD_school_shapes$schnam, p
  "PLACE @ Prescott" = "Preparatory Literary Academy of Cultural Excellence",
  "RISE Community School" = str_subset(string = OUSD_school_shapes$schnam, pattern = "I
)

```

After

```

OUSD_not_found_aliases = list(
  "Brookfield Village Elementary"      = str_subset(string = OUSD_school_shapes$schnam
  "Carl Munck Elementary"              = str_subset(string = OUSD_school_shapes$schnam
  "Community United Elementary School" = str_subset(string = OUSD_school_shapes$schnam
  "East Oakland PRIDE Elementary"      = str_subset(string = OUSD_school_shapes$schnam
  "EnCompass Academy"                 = str_subset(string = OUSD_school_shapes$schnam
  "Global Family School"               = str_subset(string = OUSD_school_shapes$schnam
  "International Community School"      = str_subset(string = OUSD_school_shapes$schnam
  "Madison Park Lower Campus"          = "Madison Park Academy TK-5",
  "Manzanita Community School"         = str_subset(string = OUSD_school_shapes$schnam
  "Martin Luther King Jr Elementary"   = str_subset(string = OUSD_school_shapes$schnam
  "PLACE @ Prescott"                  = "Preparatory Literary Academy of Cultural Exce

```

8.3. AUTOMATED TOOLS FOR STYLE AND PROJECT WORKFLOW 45

```
"RISE Community School" = str_subset(string = OUSD_school_shapes$schnam, pattern =  
)
```

3. **StyleR** - Another cool R package from the Tidyverse that can be powerful and used as a first pass on entire projects that need refactoring. The most useful function of the package is the `style_dir` function, which will style all files within a given directory. See the function's documentation and the vignette linked above for more details.
 - **Note:** The default Tidyverse styler is subtly different from some of the things we've advocated for in this document. Most notably we differ with regards to the assignment operator (`<-` vs `=`) and number of spaces before/after "tokens" (i.e. Assignment Aligner add spaces before `=` signs to align them properly). For this reason, we'd recommend the following: `style_dir(path = ..., scope = "line_breaks", strict = FALSE)`. You can also customize StyleR even more if you're really hardcore.
 - **Note:** As is mentioned in the package vignette linked above, StyleR modifies things *in-place*, meaning it overwrites your existing code and replaces it with the updated, properly styled code. This makes it a good fit on projects *with version control*, but if you don't have backups or a good way to revert back to the initial code, I wouldn't recommend going this route.
4. **Linter** - Linters are programming tools that check adherence to a given style, syntax errors, and possible semantic issues. The R linter, called `lintr`, can be found in this package. It helps keep files consistent across different authors and even different organizations. For example, it notifies you if you have unused variables, global variables with no visible binding, not enough or superfluous whitespace, and improper use of parentheses or brackets. A list of its other purposes can be found in this link, and most guidelines are based on Hadley Wickham's R Style Guide.
 - **Note:** You can customize your settings to set defaults or to exclude files. More details can be found here.
 - **Note:** The `lintr` package goes hand in hand with the `styler` package. The styler can be used to automatically fix the problems that the `lintr` catches.

Chapter 9

Working with Big Data

by Kunal Mishra and Jade Benjamin-Chung

9.1 The `data.table` package

It may also be the case that you're working with very large datasets. Generally I would define this as 10+ million rows. As is outlined in this document, the 3 main players in the data analysis space are Base R, **Tidvyerse** (more specifically, `dplyr`), and `data.table`. For a majority of things, Base R is inferior to both `dplyr` and `data.table`, with concise but less clear syntax and less speed. `Dplyr` is architected for medium and smaller data, and while its very fast for everyday usage, it trades off maximum performance for ease of use and syntax compared to `data.table`. An overview of the `dplyr` vs `data.table` debate can be found in this [stackoverflow post](#) and all 3 answers are worth a read.

You can also achieve a performance boost by running `dplyr` commands on `data.tables`, which I find to be the best of both worlds, given that a `data.table` is a special type of `data.frame` and fairly easy to convert with the `as.data.table()` function. The speedup is due to `dplyr`'s use of the `data.table` backend and in the future this coupling should become even more natural.

If you want to test whether using a certain coding approach increases speed, consider the `tictoc` package. Run `tic()` before a code chunk and `toc()` after to measure the amount of system time it takes to run the chunk. For example, you might use this to decide if you *really* need to switch a code chunk from `dplyr` to `data.table`.

9.2 Using downsampled data

In our studies with very large datasets, we save “downsampled” data that usually includes a 1% random sample stratified by any important variables, such as year or household id. This allows us to efficiently write and test our code without having to load in large, slow datasets that can cause RStudio to freeze. Be very careful to be sure which dataset you are working with and to label results output accordingly.

9.3 Optimal RStudio set up

Using the following settings will help ensure a smooth experience when working with big data. In RStudio, go to the “Tools” menu, then select “Global Options”. Under “General”:

Workspace

- **Uncheck** Restore RData into workspace at startup
- Save workspace to RData on exit – choose **never**

History

- **Uncheck** Always save history

Unfortunately RStudio often gets slow and/or freezes after hours working with big datasets. Sometimes it is much more efficient to just use Terminal / gitbash to run code and make updates in git.

Chapter 10

Data masking

by Anna Nguyen, Jade Benjamin-Chung, and Gabby Barratt Heitmann

10.1 General overview

This chapter covers data masking, a unique process in R in which columns are treated as distinct objects within their dataframe's environment. In our lab, data masking most frequently comes up when writing wrapper functions where arguments to indicate column names are supplied as strings. We often do this when we repeat the same code on multiple columns, and want to apply a function to a vector of strings that correspond to column names in a dataframe. For example, we might want to clean multiple columns using the same function or estimate the same model under different feature sets. Here, we try to break down what data masking is, why this error comes up, and common approaches to solve this problem.

10.1.1 What is Data Masking?

Within certain tidyverse operations, columns are called as if they were variables. For example, while running `df %>% mutate(X = ...)` R recognizes that `X` specifically references a column in `df` without explicitly stating its membership [`df %>% mutate(df$X = ...)`] or calling the column name as a string [`df %>% mutate("X" = ...)`].

However, this behavior may introduce errors when we attempt to incorporate variables from the global environment within these tidyverse pipelines. In the example above, `column_name = "X"` followed by `df %>% mutate(X2 = column_name + 1)` would yield an error, since `column_name` is not a column in `df` and the variable `column_name` is not defined within the environment of `df`.

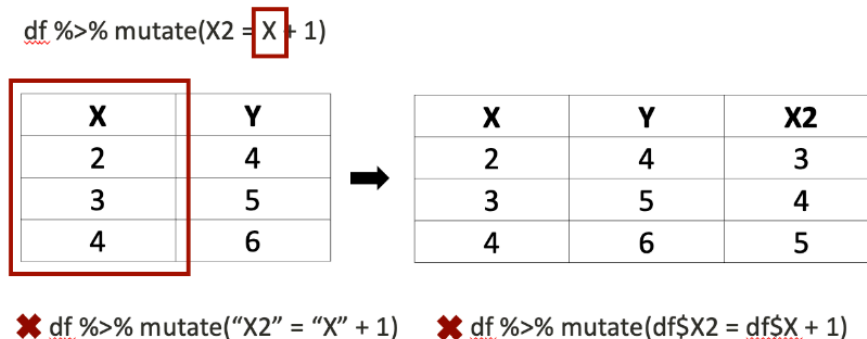


Figure 10.1: Alt text

10.1.2 Using tidy evaluation for data masking

In dplyr-based R programming, we make use of tidy evaluation. This allows us to avoid using base R syntax to reference specific columns in a data frame. By leveraging Tidy evaluation-based data masking, we can employ long pipes with several dplyr verbs to manipulate our data using stand-alone variables that store column names as strings.

For example, consider a data frame “df” that contains a column called “heavyrain” that we want to manipulate. Suppose we wanted to convert the values of “heavyrain” into a factor.

Using base R, which does not mask data, heavyrain must have quotes to be treated as a data-variable:

```
df[["outcome"]] = as.factor(df[["heavyrain"]])
```

In a dplyr pipe, heavyrain is being masked using tidy evaluation and will be correctly interpreted as a column because it is recognized as a data-variable: `df %>% mutate(outcome = as.factor(heavyrain))`

With modified data masking, heavyrain is a string that is coerced into being recognized as a data-variable:

```
var_name = "heavyrain"
df %>% mutate(outcome = as.factor(!!sym(var_name)))
```

While cleaner and often more convenient, the data frame that var_name is in is now “masked” and we refer to the vectors in the dataframe (data-variables) as though it is an object of its own (an environmental-variable). This is why we can just say the variable’s name in the context of a pipe – we treat it as though it’s an object defined in our environment. Within normal scripts, this is usually fine, because the data frame is “held on to” in the pipe. However, it

can cause some programming hurdles when writing functions that take strings of variable/column names as arguments. In the next section, we briefly describe how to troubleshoot common errors in data masking, as relevant to our lab’s work.

10.2 Technical Overview

This section covers the R functions and tools that we often use in the context of data masking, focusing on the bang bang operator (`!!`) with symbol coercion (`sym()`) and the Walrus operator (`:=`).

The combined use of `!!` and `sym()` allows us to use strings, rather than data-variables, to reference column names within dplyr. Together, `!!sym("column_name")` forces dplyr to recognize “column_name” as a data-variable prior to evaluating the rest of the expression, enabling the ability to perform calculations on the column while referring to it as a string. `sym()` is a function that turns strings into symbols. In the context of a dplyr pipe, these symbols are interpreted as data-variables. The `!!` (bang bang) operator tells dplyr to evaluate the `sym()` expression first, e.g. to unquote its expression (e.g. “column_name”) and evaluate it as a pre-existing object, first. This is helpful because often we use `sym("column_name")` within a larger expression, and dplyr might evaluate other elements of the expression first without `!!`, causing errors.

When we want to create a new column (via `mutate` or `summarize`), the Walrus operator (`:=`) allows us to specify the new column’s name using a string. For example, while `df %>% mutate("new_column" = values)` would yield an error, `df %>% mutate("new_column" := values)` will correctly create a new column called “new_column”. If we want to use a variable representing a string, we can use `!!` to force the variable to be evaluated before using `:=` to assign the value of the new column.

```
col_name = "new_column"
df %>% mutate(!!col_name := values)
```

10.2.1 Example

Suppose we want to write a function “generate_descriptive_table” to summarize how the prevalence of “outcome” varies under different levels of a “risk_factor” in a data frame “df”

We can start by writing the function shell:

```
generate_descriptive_table <- function (df, outcome, rf) {
  outcome_dist_by_rf <- ...
  return(outcome_dist_by_rf)
}
```

Next, we can filter the data frame for only rows in which “rf” and “outcome” are not missing. We can use `!!` and `sym()` within `filter` to evaluate the strings stored in “rf” and “outcome”. Note that defining `!!sym(outcome)` or `!!sym(outcome)` in variables *outside of the dplyr pipeline* will *not* work.

```
generate_descriptive_table <- function (df, outcome, rf,) {
  outcome_dist_by_rf <- df %>%
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) %>%
    ...
  return(outcome_dist_by_rf)
}
```

Similarly, we use `!!` and `sym()` in `group_by` to evaluate column name, stored as a string in the argument “rf”

```
generate_descriptive_table <- function (df, outcome, rf,) {
  outcome_dist_by_rf <- df %>%
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) %>%
    ...
  return(outcome_dist_by_rf)
}
```

Finally, we can use the walrus operator, `!!` and `sym()` with “summarize” to create a new column that takes the mean of the column referenced in “rf”. We also use “glue” or “paste” to give the new column an informative name that includes the “outcome” it describes.

```
generate_descriptive_table <- function (df, outcome, rf,) {
  outcome_dist_by_rf <- df %>%
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) %>%
    group_by(!!sym(rf)) %>%
    summarize(!!(glue::glue("{outcome}_prev")) := mean(!!sym(outcome)))
  return(outcome_dist_by_rf)
}
```

OR

```
generate_descriptive_table <- function (df, outcome, rf,) {
  outcome_dist_by_rf <- df %>%
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) %>%
    group_by(!!sym(rf)) %>%
    summarize(!!(paste0(outcome, "_prev")) := mean(!!sym(outcome)))
  return(outcome_dist_by_rf)
}
```

OR

```
generate_descriptive_table <- function (df, outcome, rf,) {
  new_column_name = paste0(outcome, "_prev")
  ...
}
```

```
outcome_dist_by_rf <- df %>%  
  filter(!is.na(sym(outcome)), !is.na(sym(rf))) %>%  
  group_by(sym(rf)) %>%  
  summarize(new_column_name := mean(sym(outcome)))  
  return(outcome_dist_by_rf)  
}
```


Chapter 11

Github

by Stephanie Djajadi and Nolan Pokpongkiat

11.1 Basics

- A detailed tutorial of Git can be found here on the CS61B website.
- If you are already familiar with Git, you can reference the summary at the end of Section B.
- If you have made a mistake in Git, you can refer to this article to undo, fix, or remove commits in git.

11.2 Github Desktop

While knowing how to use Git on the command line will always be useful since the full power of Git and its customizations and flexibility is designed for use with the command line, Github also provides Github Desktop as a graphical interface to do basic git commands; you can do all of the basic functions of Git using this desktop app. Feel free to use this as an alternative to Git on the command line if you prefer.

11.3 Git Branching

Branches allow you to keep track of multiple versions of your work simultaneously, and you can easily switch between versions and merge branches together once you've finished working on a section and want it to join the rest of your code. Here are some cases when it may be a good idea to branch:

- You may want to make a dramatic change to your existing code (called refactoring) but it will break other parts of your project. But you want

to be able to simultaneously work on other parts or you are collaborating with others, and you don't want to break the code for them.

- You want to start working on a new part of the project, but you aren't sure yet if your changes will work and make it to the final product.
- You are working with others and don't want to mix up your current work with theirs, even if you want to bring your work together later in the future.

A detailed tutorial on Git Branching can be found [here](#). You can also find instructions on how to handle merge conflicts when joining branches together.

11.4 Example Workflow

A standard workflow when starting on a new project and contributing code looks like this:

Command	Description
SETUP: FIRST TIME ONLY: <code>git clone <url></code> <code><directory_name></code>	Clone the repo. This copies all the project files in its current state on Github to your local computer.
1. <code>git pull origin master</code>	update the state of your files to match the most current version on GitHub
2. <code>git checkout -b <new_branch_name></code>	create new branch that you'll be working on and go to it
3. Make some file changes	work on your feature/implementation
4. <code>git add -p</code>	add changes to stage for commit, going through changes line by line
5. <code>git commit -m <commit message></code>	commit files with a message
6. <code>git push -u origin <branch_name></code>	push branch to remote and set to track (-u only needed if this is first push)
7. Repeat step 4-5.	work and commit often
8. <code>git push</code>	push work to remote branch for others to view
9. Follow the link given from the <code>git push</code> command to submit a pull request (PR) on GitHub online	PR merges in work from your branch into master
(10.) Your changes and PR get approved, your reviewer deletes your remote branch upon merging	
11. <code>git fetch --all --prune</code>	clean up your local git by untracking deleted remote branches

Other helpful commands are listed below.

11.5 Commonly Used Git Commands

Command	Description
<code>git clone <url> <directory_name></code>	clone a repository, only needs to be done the first time
<code>git pull origin master</code>	pull from <code>master</code> before making any changes
<code>git branch</code>	check what branch you are on
<code>git branch -a</code>	check what branch you are on + all remote branches
<code>git checkout -b <new_branch_name></code>	create new branch and go to it (only necessary when you create a new branch)
<code>git checkout <branch name></code>	switch to branch
<code>git add <file name></code>	add file to stage for commit
<code>git add -p</code>	adds changes to commit, showing you changes one by one
<code>git commit -m <commit message></code>	commit file with a message
<code>git push -u origin <branch_name></code>	push branch to remote and set to track (-u only works if this is first push)
<code>git branch --set-upstream-to origin <branch_name></code>	set upstream to <code>origin/<branch_name></code> (use if you forgot -u on first push)
<code>git push origin <branch_name></code>	push work to branch
<code>git checkout <branch_name> git merge master</code>	switch to branch and merge changes from <code>master</code> into <code><branch_name></code> (two commands)
<code>git merge <branch_name> master</code>	switch to branch and merge changes from <code>master</code> into <code><branch_name></code> (one command)
<code>git checkout --track origin/<branch_name></code>	pulls a remote branch and creates a local branch to track it (use when trying to pull someone else's branch onto your local computer)
<code>git push --delete <remote_name> <branch_name></code>	delete remote branch
<code>git branch -d <branch_name></code>	deletes local branch, -D to force
<code>git fetch --all --prune</code>	untrack deleted remote branches

11.6 How often should I commit?

It is good practice to commit every 15 minutes, or every time you make a significant change. It is better to commit more rather than less.

11.7 What should be pushed to Github?

Never push .Rout files! If someone else runs an R script and creates an .Rout file at the same time and both of you try to push to github, it is incredibly difficult to reconcile these two logs. If you run logs, keep them on your own system or (preferably) set up a shared directory where all logs are name and date timestamped.

There is a standardized `.gitignore` for R which you can download and add to your project. This ensures you're not committing log files or things that would otherwise best be left ignored to GitHub. This is a great discussion of project-oriented workflows, extolling the virtues of a self-contained, portable projects, for your reference.

Chapter 12

Unix commands

by Stephanie Djajadi, Kunal Mishra, Anna Nguyen, and Jade Benjamin-Chung
We typically use Unix commands in Terminal (for Mac users) or Git Bash (for Windows users) to

1. Run a series of scripts in parallel or in a specific order to reproduce our work
2. To check on the progress of a batch of jobs
3. To use git and push to github

12.1 Basics

On the computer, there is a desktop with two folders, `folder1` and `folder2`, and a file called `file1`. Inside `folder1`, we have a file called `file2`. Mac users can run these commands on their terminal; it is recommended that Windows users use Git Bash, not Windows PowerShell.

12.2 Syntax for both Mac/Windows

When typing in directories or file names, quotes are necessary if the name includes spaces.

Command	Description
<code>cd desktop/folder1</code>	Change directory to <code>folder1</code>
<code>pwd</code>	Print working directory
<code>ls</code>	List files in the directory
<code>cp "file2" "newfile2"</code>	Copy file (remember to include file extensions when typing in file names like <code>.pdf</code> or <code>.R</code>)

Command	Description
<code>mv "newfile2" "file3"</code>	Rename <code>newfile2</code> to <code>file3</code>
<code>cd ..</code>	Go to parent of the working directory (in this case, <code>desktop</code>)
<code>mv "file1" folder2</code>	Move <code>file1</code> to <code>folder2</code>
<code>mkdir folder3</code>	Make a new folder in <code>folder2</code>
<code>rm <filename></code>	Remove files
<code>rm -rf folder3</code>	Remove directories (<code>-r</code> will attempt to remove the directory recursively, <code>-rf</code> will force removal of the directory)
<code>clear</code>	Clear terminal screen of all previous commands

12.3 Running Bash Scripts

Windows	Mac / Linux	Description
<code>chmod +750 <filename.sh></code>	<code>chmod +x <filename.sh></code>	Change access permissions for a file (only needs to be done once)
<code>./<filename.sh></code>	<code>./<filename.sh></code>	Run file (<code>./</code> to run any executable file)
<code>bash</code>	<code>bash</code>	Run shell script in the background
<code>bash_script_name.sh</code>	<code>bash_script_name.sh &</code>	
<code>&</code>		

12.4 Running Rscripts in Windows

Note: This code seems to work only with Windows Command Prompt, not with Git Bash.

When R is installed, it comes with a utility called Rscript. This allows you to run R commands from the command line. If Rscript is in your PATH, then typing Rscript into the command line, and pressing enter, will not error. Otherwise, to use Rscript, you will either need to add it to your PATH (as an environment variable), or append the full directory of the location of Rscript on your machine. To find the full directory, search for where R is installed your computer. For instance, it may be something like below (this will vary depending on what version of R you have installed):

```
C:\Program Files\R\R-3.6.0\bin
```

For appending the PATH variable, please view this link. I strongly recommend completing this option.

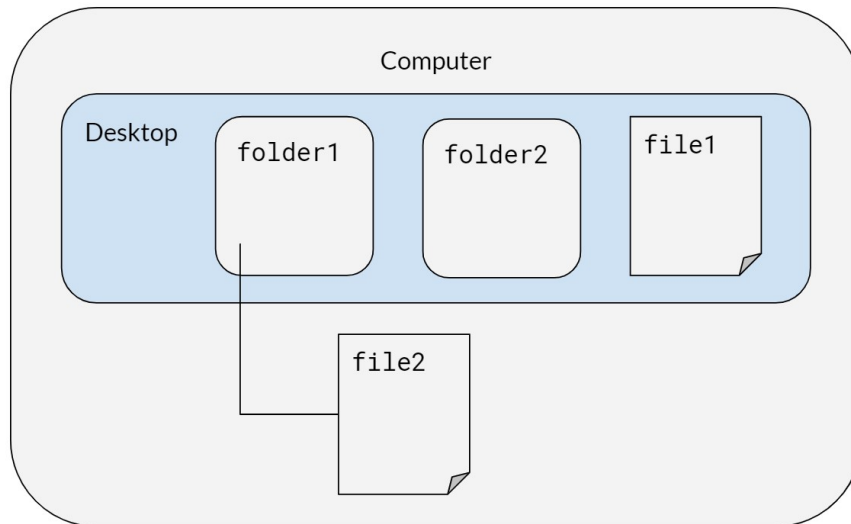


Figure 12.1: Here is our example desktop.

If you add the PATH as an environment variable, then you can run this line of code to test: `Rscript -e "cat('this is a test')"`, where the `-e` flag refers to the expression that will be executed.

If you do not add the PATH as an environment variable, then you can run this line of code to replicate the results from above: `"C:\Program Files\R\R-3.6.0\bin" -e "cat('this is a test')"`

To run an R script from the command line, we can say: `Rscript -e "source('C:/path/to/script/some_code.R')"`

12.4.1 Common Mistakes

- Remember to include all of the quotation marks around file paths that have a spaces.
- If you attempt to run an R script but run into `Error: '\U' used without hex digits in character string starting "C:\U"`, try replacing all `\` with `\\` or `/`.

12.5 Checking tasks and killing jobs

```

MINGW64/c/Users/Stephanie Djajadi/desktop/folder2
Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~
$ cd ~/desktop

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop
$ pwd
/c/Users/Stephanie Djajadi/desktop

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop
$ ls
desktop.ini  file1.txt  folder1/  folder2/

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop
$ cd folder1

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder1
$ ls
file2.txt

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder1
$ cp file2.txt newfile2.txt

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder1
$ ls
file2.txt  newfile2.txt

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder1
$ mv newfile2.txt file3.txt

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder1
$ ls
file2.txt  file3.txt

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder1
$ cd ..

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop
$ ls
desktop.ini  file1.txt  folder1/  folder2/

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop
$ mv file1.txt folder2

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop
$ cd folder2

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder2
$ ls
file1.txt

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder2
$ mkdir folder3

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder2
$ ls
file1.txt  folder3/

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder2
$ rm file1.txt

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder2
$ rm -rf folder3

Stephanie Djajadi@DESKTOP-L0H5V00 MINGW64 ~/desktop/folder2
$ ls

```

Figure 12.2: Here is an example of what your terminal might look like after executing the commands in the order listed above.

Windows	Mac / Linux	Description
<code>tasklist</code>	<code>ps -v</code>	List all processes on the command line
	<code>top -o [cpu/rsize]</code>	List all running processes, sorted by CPU or memory usage
<code>taskkill /F /PID pid_number</code>	<code>kill <PID_number></code>	Kill a process by its process ID
<code>taskkill /IM "process name" /F</code>		Kill a process by its name
<code>start /b program.exe</code>		Runs jobs in the background (exclude <code>/b</code> if you want the program to run in a new console)
	<code>nohup</code>	Prevents jobs from stopping
	<code>disown</code>	Keeps jobs running in the background even if you close R
<code>taskkill /?</code>		Help, lists out other commands

To kill a task in Windows, you can also go to Task Manager > More details > Select your desired app > Click on End Task.

12.6 Running big jobs

For big data workflows, the concept of “backgrounding” a bash script allows you to start a “job” (i.e. run the script) and leave it overnight to run. At the top level, a bash script (`0-run-project.sh`) that simply calls the directory-level bash scripts (i.e. `0-prep-data.sh`, `0-run-analysis.sh`, `0-run-figures.sh`, etc.) is a powerful tool to rerun every script in your project. See the included example bash scripts for more details.

- **Running Bash Scripts in Background:** Running a long bash script is not trivial. Normally you would run a bash script by opening a terminal and typing something like `./run-project.sh`. But what if you leave your computer, log out of your server, or close the terminal? Normally, the bash script will exit and fail to complete. To run it in background, type `./run-project.sh &; disown`. You can see the job running (and CPU utilization) with the command `top` or `ps -v` and check your memory with `free -h`.

Alternatively, to keep code running in the background even when an SSH connection is broken, you can use `tmux`. In terminal or gitbash follow the steps below. This site has useful tips on using `tmux`.

```
# create a new tmux session called session_name
tmux new -ssession_name

# run your job of interest
R CMD BATCH myjob.R &

# check that it is running
ps -v

# to exit the tmux session (Mac)
ctrl + b
d

# to reopen the tmux session to kill the job or
# start another job
tmux attach -tsession_name
```

- **Deleting Previously Computed Results:** One helpful lesson we've learned is that your bash scripts should remove previous results (computed and saved by scripts run at a previous time) so that you never mix results from one run with a previous run. This can happen when an R script errors out before saving its result, and can be difficult to catch because your previously saved result exists (leading you to believe everything ran correctly).
- **Ensuring Things Ran Correctly:** You should check the `.Rout` files generated by the R scripts run by your bash scripts for errors once things are run. A utility file is include in this repository, called `runFileSaveLogs`, and is used by the example bash scripts to... run files and save the generated logs. It is an awesome utility and one I definitely recommend using. Before using `runFileSaveLogs`, it is necessary to put the file in the home working directory. For help and documentation, you can use the command `./runFileSaveLogs -h`. See example code and example usage for `runFileSaveLogs` below.

12.6.1 Example code for `runfileSaveLogs`

```
#!/usr/bin/env python3
# Type "./runFileSaveLogs -h" for help

import os
import sys
```



```

import argparse
import getpass
import datetime
import shutil
import glob
import pathlib

# Setting working directory to this script's current directory
os.chdir(os.path.dirname(os.path.abspath(__file__)))

# Setting up argument parser
parser = argparse.ArgumentParser(description='Runs the argument R script(s) - in parallel if specified')

# Function ensuring that the file is valid
def is_valid_file(parser, arg):
    if not os.path.exists(arg):
        parser.error("The file %s does not exist!" % arg)
    else:
        return arg

# Function ensuring that the directory is valid
def is_valid_directory(parser, arg):
    if not os.path.isdir(arg):
        parser.error("The specified path (%s) is not a directory!" % arg)
    else:
        return arg

# Additional arguments that can be added when running runFileSaveLogs
parser.add_argument('-p', '--parallel', action='store_true', help="Runs the argument R scripts in parallel")
parser.add_argument("-i", "--identifier", help="Adds an identifier to the directory name where the logs are saved")
parser.add_argument('filenames', nargs='+', type=lambda x: is_valid_file(parser, x))

args = parser.parse_args()
args_dict = vars(args)

print(args_dict)

# Run given R Scripts
for filename in args_dict["filenames"]:
    system_call = "R CMD BATCH" + " " + filename
    if args_dict["parallel"]:
        system_call = "nohup" + " " + system_call + " &"

    os.system(system_call)

```

```

# Create the directory (and any parents) of the log files
currentUser = getpass.getuser()
currentTime = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
logDirPrefix = "/home/kaiserData/logs/" # Change to the directory where the logs should be
logDir = logDirPrefix + currentTime + "-" + currentUser

# If specified, adds the identifier to the filename of the log
if args.identifier is not None:
    logDir += "-" + args.identifier

logDir += "/"

pathlib.Path(logDir).mkdir(parents=True, exist_ok=True)

# Find and move all logs to this new directory
currentLogPaths = glob.glob('/*.Rout')

for currentLogPath in currentLogPaths:
    filename = currentLogPath.split("/")[-1]
    shutil.move(currentLogPath, logDir + filename)

```

12.6.2 Example usage for runfileSaveLogs

This example bash script runs files and generates logs for five scripts in the `kaiserflu/3-figures` folder. Note that the `-i` flag is used as an identifier to add `figures` to the filename of each log.

```

#!/bin/bash

# Copy utility run script into this folder for concision in call
cp ~/kaiserflu/runFileSaveLogs ~/kaiserflu/3-figures/

# Run folder scripts and produce output
cd ~/kaiserflu/3-figures/
./runFileSaveLogs -i "figures" \
fig-mean-season-age.R \
fig-monthly-rate.R \
fig-point-estimates-combined.R \
fig-point-estimates.R \
fig-weekly-rate.R

# Remove copied utility run script
rm runFileSaveLogs

```

Chapter 13

Reproducible Environments

by Anna Nguyen

13.1 Package Version Control with `renv`

13.1.1 Introduction

Replicable code should produce the same results, regardless of when or where it's run. However, our analyses often leverage open-source R packages that are developed by other teams. These packages continue to be developed after research projects are completed, which may include changes to analysis functions that could impact how code runs for both other team members and external replicators.

For example, suppose we had used a function that took in one argument, such that our code contained `example_function(arg_a = "a")`. A few months after we publish our code, the package developers update the function to take in another mandatory argument `arg_b`. If someone runs our code, but has the most recent version of the package, they'll receive an error message that the argument `arg_b` is missing and will not be able to full reproduce our results.

To ensure that the right functions are used in replication efforts, it is important for us to keep track of package versions used in each project.

`renv` can be to promote reproducible environments within R projects. `renv` creates individual package libraries for each project instead of having all projects, which may use different versions of the same package, share the same package library. However, for projects that use many packages, this process can be memory intensive and increase the time needed for a new users to start running code.

In this lab manual chapter, we provide a quick tutorial for integrating `renv`

into research workflows. For more detailed instructions, please refer to the **renv** package vignette.

13.1.2 Implementing **renv** in projects

Ideally, **renv** should be initiated at the start of projects and updated continuously when new packages are introduced in the codebase. However, this process can be initiated at any point in a project

To add **renv** to your workflow, follow these steps:

1. Install the **renv** package by running `install.packages("renv")`
2. Create an RProject file and ensure that your working directory is set to the correct folder
3. In the R console, run `renv::init()` to initialize **renv** in your R Project
4. This will create the following files: `renv.lock`, `.Rprofile`, `renv/settings.json` and `renv/activate.R`. Commit and push these files to GitHub so that they're accessible to other users.
5. As you write code, update the project's R library by running `renv::snapshot()` in the R console
6. Add `renv::restore()` to the head of your config file, to make sure that all users that run your code are on the same package versions.

13.1.3 Using projects with **renv**

If you're starting to work on an ongoing project that already has **renv** set up, follow these steps to ensure that you're using the same project versions.

1. Install the **renv** package by running `install.packages("renv")`
2. Pull the most updated version of the project from GitHub
3. Open the project's RProject file
4. Run `renv::restore()`. In our lab's projects, this is often already found at the top of the config file, so you can just run scripts as is.
5. This will pull up a list of the project's packages that need to be updated for you to be consistent with the project. The console will ask if you want to proceed with updating these packages - type "Y" to continue.
6. Wait for the correct versions of each package to install/update. This may take some time, depending on how many packages the project uses.
7. Your R environment should now be using the same package versions as specified in the **renv** lock file. You should now be able to replicate the code.
8. If you make edits to the code and introduce new/updated packages, see the section above for instructions on how to make updates.

Chapter 14

Code Publication

by Nolan Pokpongkiat

14.1 Checklist overview

1. Fill out file headers
2. Clean up comments
3. Document functions
4. Remove deprecated filepaths
5. Ensure project runs via bash
6. Complete the README
7. Clean up feature branches
8. Create Github release

14.2 Fill out file headers

Every file in a project should have a header that allows it to be interpreted on its own. It should include the name of the project and a short description for what this file (among the many in your project) does specifically. See [template here](#).

14.3 Clean up comments

Make sure comments in the code are for code documentation purposes only. Do not leave comments to self in the final script files.

14.4 Document functions

Every function you write must include a header to document its purpose, inputs, and outputs. See [template for the function documentation here](#).

14.5 Remove deprecated filepaths

All file paths should be defined in `0-config.R`, and should be set relative to the project working directory. All absolute file paths from your local computer should be removed, and replaced with a relative path. If a third party were to re-run this analysis, if they need to download data from a separate source and change a filepath in the `0-config.R` to match, make sure to specify in the README which line of `0-config.R` needs to be substituted.

14.6 Ensure project runs via bash

The project should be configured to be entirely reproducible by running a master bash script, `run-project.sh`, which should live at the top directory. This bash script can call other bash scripts in subfolders, if necessary. Bash scripts should use the `runFileSaveLogs` utility script, which is a wrapper around the `Rscript` command, allowing you to specify where `.Rout` log files are moved after the R scripts are run.

See [usage and documentation here](#).

14.7 Complete the README

A `README.md` should live at the top directory of the project. This usually includes a Project Overview and a Directory Structure, along with the names of the contributors and the Creative Commons License. See below for a template:

Overview

To date, coronavirus testing in the US has been extremely limited. Confirmed COVID-19 case counts underestimate the total number of infections in the population. We estimated the total COVID-19 infections – both symptomatic and asymptomatic – in the US in March 2020. We used a semi-Bayesian approach to correct for bias due to incomplete testing and imperfect test performance.

Directory structure

- `0-config.R`: configuration file that sets data directories, sources base functions, and loads required libraries
- `0-base-functions`: folder containing scripts with functions used in the analysis

- 0-base-functions.R: R script containing general functions used across the analysis
- 0-bias-corr-functions.R: R script containing functions used in bias correction
- 0-bias-corr-functions-undertesting.R: R script containing functions used in bias correction to estimate the percentage of underestimation due to incomplete testing vs. imperfect test accuracy
- 0-prior-functions.R: R script containing functions to generate priors
- 1-data: folder containing data processing scripts NOTE: some scripts are deprecated
- 2-analysis: folder containing analysis scripts. To rerun all scripts in this subdirectory, run the bash script 0-run-analysis.sh.
 - 1-obtain-priors-state.R: obtain priors for each state
 - 2-est-expected-cases-state.R: estimate expected cases in each state
 - 3-est-expected-cases-state-perf-testing.R: estimate expected cases in each state, estimate the percentage of underestimation due to incomplete testing vs. imperfect test accuracy
 - 4-obtain-testing-protocols.R: find testing protocols for each state.
 - 5-summarize-results.R: summarize results; obtain results for in text numerical results.
- 3-figure-table-scripts: folder containing figure scripts. To rerun all scripts in this subdirectory, run the bash script 0-run-figs.sh.
 - 1-fig-testing.R: creates plot of testing patterns by state over time
 - 2-fig-cases-usa-state-bar.R: creates bar plot of confirmed vs. estimated infections by state
 - 3a-fig-map-usa-state.R: creates map of confirmed vs. estimated infections by state
 - 3b-fig-map-usa-state-shiny.R: creates map of confirmed vs. estimated infections by state with search functionality by state
 - 4-fig-priors.R: creates figure with priors for US as a whole

- 5-fig-density-usa.R: creates figure of distribution of estimated cases in the US
- 6-table-data-quality.R: creates table of data quality grading from COVID Tracking Project
- 7-fig-testpos.R: creates figure of the probability of testing positive among those tested by state
- 8-fig-percent-underesting-state.R: creates figure of the percentage of under estimation due to incomplete testing
- 4-figures: folder containing figure files.
- 5-results: folder containing analysis results objects.
- 6-sensitivity: folder containing scripts to run the sensitivity analyses

Contributors: Jade Benjamin-Chung, Sean L. Wu, Anna Nguyen, Stephanie Djajadi, Nolan N. Pokpongkiat, Anmol Seth, Andrew Mertens

Wu SL, Mertens A, Crider YS, Nguyen A, Pokpongkiat NN, Djajadi S, et al. Substantial underestimation of SARS-CoV-2 infection in the United States due to incomplete testing and imperfect test accuracy. medRxiv. 2020; 2020.05.12.20091744. doi:10.1101/2020.05.12.20091744

When possible, also include a description of the RDS results that are generated, detailing what data sources were used, where the script lives that creates it, and what information the RDS results hold.

14.8 Clean up feature branches

In the remote repository on Github, all feature branches aside from master should be merged in and deleted. All outstanding PRs should be closed.

14.9 Create Github release

Once all of these items are verified, create a tag to make a Github release, which will tag the repository, creating a marker at this specific point in time.

Detailed instructions here.

Chapter 15

Data Publication

Adapted from Fanice Nyatigo and Ben Arnold's chapter in the Proctor-UCSF Lab Manual

15.1 Overview

Warning! NEVER push a dataset into the public domain (e.g., GitHub, OSF) without first checking with Jade to ensure that it is appropriately de-identified and we have approval from the sponsor and/or human subjects review board to do so. For example, we will need to re-code participant IDs (even if they contain no identifying information) before making data public to completely break the link between IDs and identifiable information stored on our servers.

If you are releasing data into the public domain, then consider making available *at minimum* a `.csv` file and a codebook of the same name (note: you should have a codebook for internal data as well). We often also make available `.rds` files as well. For example, your `mystudy/data/public` directory could include three files for a single dataset, two with the actual data in `.rds` and `.csv` formats, and a third that describes their contents:

```
analysis_data_public.csv  
analysis_data_public.rds  
analysis_data_public_codebook.txt
```

In general, datasets are usually too big to save on GitHub, but occasionally they are small. Here is an example of where we actually pushed the data directly to GitHub: <https://github.com/ben-arnold/enterics-seroepi/tree/master/data> .

If the data are bigger, then maintaining them under version control in your git repository can be unweildy. Instead, we recommend using another stable repository that has version control, such as the Open Science Framework (osf.io). For example, all of the data from the WASH Benefits trials (led by investigators at Berkeley, icddr,b, IPA-Kenya and others) are all stored through data components nested within in OSF projects: <https://osf.io/tprw2/>. Another good option is Dryad (datadryad.org) or the (Stanford Digital Repository)[<https://sdr.stanford.edu/>].

We recommend cross-linking public files in GitHub (scripts/notebooks only) and OSF/Dryad/Stanford Digital Repository.

Below are the main steps to making data public, after finalizing the analysis datasets and scripts:

1. Remove Protected Health Information (PHI)
2. Create public IDs or join already created public IDs to the data
3. Create an OSF repository and/or Dryad/Stanford Digital Repository
4. Edit analysis scripts to run using the public datasets and test (optional)
5. Create a public github page for analysis scripts and link to OSF and/or Dryad/Zenodo
6. Go live

15.2 Removing PHI

Once the data is finalized for analysis, the first step is to strip it of Protected Health Information (PHI), or any other data that could be used to link back to specific participants, such as names, birth dates, or GPS coordinates at the village/neighborhood level or below. PHI includes, but is not limited to:

15.2.1 Personal information

These are identifiers that directly point to specific individuals, such as:

- Names, addresses, photographs, date of birth
- A combination of age, sex, and geographic location (below population 20,000) is considered identifiable

15.2.2 Dates

Any specific dates (e.g., study visit dates, birth dates, treatment dates) are usually problematic.

- If a dataset requires high resolution temporal information, coarsen visit or measurement dates to be two variables: year and week of the year (1-52).
 - If a dataset requires age, provide that information without a birth date (typically month resolution is sufficient)
-

Caution! *If making changes to the format of dates or ages, make sure your analysis code runs on these modified versions of the data (step 3)!*

15.2.3 Geographic information

Do not include GPS coordinates (longitude, latitude) except in special circumstances where they have been obfuscated/shifted. Reach out to Jade before doing this because it can be complicated.

Do not include place names or codes (e.g., US Zip Codes) if the place contains <20,000 people. For villages or neighborhoods, code them with uninformative IDs. For sub-districts or districts, names are fine.

If an analysis requires GPS locations (e.g., to make a map), then typically we include a disclaimer in the article's data availability statement that explains we cannot make GPS locations public to protect participant confidentiality. As a middle ground, we typically make our *code* public that runs on the geo-located data for transparency, even if independent researchers can't actually run that code (although please be careful to ensure the code itself does not in any way include geographic identifiers).

For more examples of what constitutes PHI, please refer to this link: <https://cphs.berkeley.edu/hipaa/hipaa18.html>

15.3 Create public IDs

15.3.1 Rationale

The Stanford IRB requires that public datasets not include the original study IDs to identify participants or other units in the study (such as village IDs). The reason is that those IDs are linked in our private datasets to PHI. By creating a new set of public IDs, the public dataset is one step further removed from the potential to link to PHI.

15.3.2 A single set of public IDs for each study

For each study, it is ideal to create a single set of public IDs whenever possible. We could create a new set of public IDs for every public dataset, but the downside is that independent researchers could no longer link data that might be related. By creating a single set of public IDs associated with each internal study ID, public files retain the link.

Maintaining a single set of public IDs requires a shared “bridge” dataset, that includes a row for each study ID and has the associated public ID. For studies

with multiple levels of ID, we would typically have separate bridge datasets for each type of ID (e.g., cluster ID, participant ID, etc.)

Create a public ID that can be used to uniquely identify participants and that can internally be linked to the original study IDs. We recommend creating a subdirectory in the study's shared data directory to store the public IDs. The shared location enables multiple projects to use the same IDs. Create the IDs using a script that reads in the study IDs, creates a unique (uninformative) public ID for the study IDs, and then saves the bridge dataset. The script should be saved in the same directory as the public ID files.

Caution! Note that small differences may arise if the new public IDs do not necessarily order participants in the same way as the internal IDs. The small differences are all in estimates that rely on resampling, such as Bootstrap CIs, permutation P-values, and TMLE, as the resampling process may lead to slightly different re-samples. The key here, to ensure the results are consistent irrespective of the dataset used, is simply to not assign public IDs randomly. Use `rank()` on the internal ID instead of `row_number()` to ensure that the order is always the same.

15.3.3 Example scripts

We have created a self-contained and reproducible example that you can run and replicate when making data public for your projects. It contains the following files and folders:

1. `data/final/-` folder containing the projects final data in both csv and rds formats
2. `code/DEMO_generate_public_IDs.R`- creates randomly generated public IDs that can be matched to the trial's assigned patient IDs.
3. `data/make_public/DEMO_internal_to_publicID.csv`- the output from step #2, a bridge dataset with two variables- the new public ID and the patient's assigned ID.
4. `code/DEMO_create_public_datasets.R`- joins the public IDs to the trial's full dataset, and strips it of the assigned patient ID.
5. `data/public/-` folder containing the output from step #3- de-identified public dataset, in csv and rds formats, with uniquely identifying public IDs that cannot be easily linked back to the patient's ID.

The example workflow is accessible via GitHub: <https://github.com/proctor-ucsf/dcc-handbook/tree/master/templates/making-data-public>

15.4 Create a data repository

First, ensure that you create a codebook and metadata file for each public dataset. See the DCC guide on Documenting datasets. Use the same name as the datasets, but with “-codebook.txt” / “-codebook.html” / “-codebook.csv” at the end (depending on the file format for the codebook). One nice option is the R codebook package, which also generates JSON output that is machine-readable.

15.4.1 Steps for creating an Open Science Framework (OSF) repository:

1. Create a new OSF project per these instructions: <https://help.osf.io/article/252-create-a-project>
2. Create a data component and upload the datasets in .csv and .rds format along with the codebooks. The primary format for public dissemination is .csv but we make the .rds files available too as auxiliary files for convenience.
3. Create a notebook component and upload the final .html files (which will not be on github... but see optional item below)
4. On the OSF landing Wiki, provide some context. Here is a recent example: <https://osf.io/954bt/>
5. Create a Digital Object Identifier (DOI) for the repository. A DOI is a unique identifier that provides a persistent link to content, such as a dataset in this case. Learn more about DOIs
6. Optional: Complete the software checklist and system requirement guide for the analysis to guide others. Include it on the GitHub README for the project: <https://github.com/proctor-ucsf/mordor-antibody>

15.5 Edit and test analysis scripts

Make minor changes to the analysis scripts so that they run on public data. If using version control in GitHub, the most straight-forward way is to create a branch from the main git branch that reads in the public files, and then renames the new public ID variable, e.g., “id_public” to the internally recognized ID variable name, e.g. “recordID”, when reading in the public data. Re-run all the analysis scripts to ensure that they still work with the public version of the dataset.

15.6 Create a public GitHub page for public scripts

At minimum, we should include all of the scripts required to run the analyses. **IMPORTANT:** ensure you have taken a snapshot and saved your computing

environment using the `renv` package (`renv`).

See examples:

- ACTION - <https://github.com/proctor-ucsf/ACTION-public>
- NAITRE - <https://github.com/proctor-ucsf/NAITRE-primary>

Caution! *Read through the scripts carefully to ensure there is no PHI in the code itself*

Once a public GitHub page exists, you can create a new component on an OSF project (step 3, above) and link it to the public version of the GitHub repo.

15.7 Go live

On GitHub, it is useful to create an official “release” version to freeze the repository, where you can have “associated files” with each version. Include the .html notebook output as additional files — since they aren’t tracked in GitHub, it does provide a way of freezing / saving the HTML output for us and others. OSF examples of a studies from UCSF’s Proctor Foundation:

- ACTION - <https://osf.io/ca3pe/>
- NAITRE - <https://osf.io/ujeyb/>
- MORDOR Niger antibody study - <https://osf.io/dgsq3/>

Further reading on end-to-end data management: How to Store and Manage Your Data - PLOS

Chapter 16

Slurm and cluster computing on ROAR

by Jess Grembi

Many of the bioinformatic tools that we use require a large amount of RAM, rely on large databases, and can be run in parallel across many samples. For these types of analyses, we use ROAR Collab, Penn State’s computing cluster. ROAR uses Slurm, an open source, scalable cluster management and job scheduling system for computing clusters. You will need to request an account, and once you have one Jess can add you to the lab account to access our storage areas and compute resources. Please refer to the ROAR User Guide to learn about the system and how to use it. Below, we include a few tips specific to how we use ROAR Collab in our lab.

16.1 Getting started

To access ROAR Collab, in terminal, log in using the following syntax and replace , with your PSU alias. You will be prompted to enter your PSU password (the same one you use for your email and other accounts) and to complete two-factor authentication.

```
ssh <user>@submit.hpc.psu.edu
```

Once you log in, you can view the contents of your home directory in command line by entering `cd $HOME`. You can create subfolders within this directory using the `mkdir` command. For example, you could make a “code” subdirectory and clone a Github repository there using the following code:

```
cd $HOME
mkdir code
```

```
git clone https://github.com/jadebc/covid19-infections.git
```

16.1.1 Storage

Each user account is provided with 3 different storage assets. The `$HOME` directory is a good place to store code and small test files (quota: 16 GB per user) and the `$WORK` directory is a good place to store larger items that you will use regularly (quota: 128 GB per user). Save intermediate processing files and temporary files generated by specific bioinformatic tools to the `$SCRATCH` directory (quota: unlimited, but files are automatically deleted 30 days after their last modification). You can read more about storage options on ROAR [here](#).

The lab has also purchased active storage space (45 TiB, accessible at `/storage/group/jag548/default`) and archive storage (15 TiB, accessible at `/storage/archive/jag548/default`). Archive storage is about 1/4 the cost of active storage so raw sequencing files should be moved to the archive storage once you've completed the preliminary data processing steps and are fairly certain you will not be needing them again anytime soon.

You can check the use of your personal storage and the group resources easily by running:

```
check_quota
```

16.1.2 Submitting jobs

ROAR has various CPU and GPU compute resources. On the basic cluster, all users have access to the following resources (across all jobs they have submitted):

- 48h runtime
- 800 GB
- 100 cores

There are 3 different hardware partitions, and you need to specify which you want your jobs to run on:

- basic cores: 4GB/core
- standard cores: 8GB/core
- high memory cores: 20GB/core

The task/job being run should dictate which partition you should use.

Your credit account "jag548_cr_credit" is active. To use the account, you will need to specify it in your job submission by either selecting it via the drop down options on the portal, or by using the directive "`-account= jag548_cr_credit`". Additionally, you will also need to specify the desired hardware partition for your job to launch correctly. The options are "basic", "standard", and "himem".

16.1.3 One-Time System Set-Up

To keep the install packages consistent across different nodes, you will need to explicitly set the pathway to your R library directory.

Open your `~/.Renviron` file (`vi ~/.Renviron`) and append the following line:

Note: Once you open the file using `vi [file_name]`, you must press `i` (on Mac OS) or `Insert` (on Windows) to make edits. After you finish, hit `Esc` to exit editing mode and type `:wq` to save and close the file.

```
R_LIBS=~R/x86_64-pc-linux-gnu-library/4.0.2
```

Alternatively, run an R script with the following code on ROAR Collab:

```
r_environ_file_path = file.path(Sys.getenv("HOME"), ".Renviron")
if (!file.exists(r_environ_file_path)) file.create(r_environ_file_path)

cat("\nR_LIBS=~R/x86_64-pc-linux-gnu-library/4.0.2",
    file = r_environ_file_path, sep = "\n", append = TRUE)
```

To load packages that run off of C++, you'll need to set the correct compiler options in your R environment.

Open the Makevars file in ROAR Collab (`vi ~/.R/Makevars`) and append the following lines

```
CXX14FLAGS=-O3 -march=native -mtune=native -fPIC
CXX14=g++
```

Alternatively, create an R script with the following code, and run it on ROAR Collab:

```
dotR = file.path(Sys.getenv("HOME"), ".R")
if (!file.exists(dotR)) dir.create(dotR)

M = file.path(dotR, "Makevars")
if (!file.exists(M)) file.create(M)

cat("\nCXX14FLAGS=-O3 -march=native -mtune=native -fPIC",
    "CXX14=g++",
    file = M, sep = "\n", append = TRUE)
```

16.2 Moving files to ROAR Collab

For this reason, it's best to create a bash script that records the file transfer process for a given project. See example code below:

```
# note: the following steps should be done from your local
# (not after ssh-ing into ROAR Collab)
```

```

# securely transfer folders from Box to ROAR Collab work directory
# note: the -r option is for folders and is not needed for files
scp -r "Box/malaria-project/folder-1/" USERNAME@submit.hpc.psu.edu:/work/

# securely transfer folders from Box to your ROAR Collab scratch directory
scp -r "Box/malaria-project/folder-2/" USERNAME@submit.hpc.psu.edu:/scratch/

# securely transfer folders from Box to our shared storage directory
scp -r "Box/malaria-project/folder-3/" USERNAME@submit.hpc.psu.edu:/storage/group/jag5

```

16.3 Installing packages on Sherlock

When you begin working on Sherlock, you will most likely encounter problems with installing packages. To install packages, login to Sherlock on the command line and open a development node using the command `sdev`. Do not attempt to do this in the RStudio Server (see next section), as you will have to re-do it for every new session you open.

```
ssh USERNAME@login.sherlock.stanford.edu
```

```
sdev
```

There is a package installation file explicitly written for Sherlock that you should run before testing any code and sourcing the configuration file. You should only have to install packages once. Sherlock requires that you specify the repository where the package is downloaded from. You may also need to add an additional argument to `install.packages` to prevent the packages from locking after installation:

```
install.packages(<PACKAGE NAME>, repos="http://cran.us.ur-project.org",
  INSTALL_opts = "--no-lock")
```

In order for some R packages to work on Sherlock, it is necessary to load specific software modules before running R. These must be loaded in Sherlock each time you want to use the package in R. For example, for spatial and random effects analyses, you may need the modules/packages below. These modules must also be loaded on the command line prior to opening R in order for package installation to work.

```

module --force purge # remove any previously loaded modules, including math and devel
module load math
module load math gmp/6.1.2
module load devel
module load gcc/10
module load system
module load json-glib/1.4.4

```

```

module load curl/7.81.0
module load physics
module load physics udunits geos
module load physics gdal/2.2.1 # for R/4.0.2
module load physics proj/4.9.3 # for R/4.0.2
module load pandoc/2.7.3

module load R/4.0.2

```

R # Open R in the Shell window to install individual packages or test code

Rscript install-packages-sherlock.R # Alternatively, run the entire package installation script

Figuring out the issues with some packages will require some trial and error. If you are still encountering problems installing a package, you may have to install other dependencies manually by reading through the error messages. If you try to install a dependency from CRAN and it isn't working, it may be a module. You can search for it using the `module spider` command:

```
module spider DEPENDENCY NAME
```

However, you can also reach out to the Sherlock team for help. You can email them at srcc-support@stanford.edu. They also hold office hours.

16.4 Testing your code

Both of the following ways to test code on Sherlock are recommended for making small changes, such as editing file paths and making sure the packages and source files load. You should write and test the functionality of your script locally, only testing on Sherlock once major bugs are out.

16.4.1 The command line

There are two main ways to explore and test code on Sherlock. The first way is best for users who are comfortable working on the command line and editing code in base R. Even if you are not comfortable yet, this is probably the better way because these commands will transfer between Sherlock and other cluster computers using Slurm.

Typically, you will want to initially test your scripts by initiating a development node using the command `sdev`. This will allocate a small amount of computing resources for 1 hour. You can access R via command line using the following code.

```

# open development node
sdev

```

Load all the modules required by the packages you are using

```

module load MODULE NAME

# Load R (default version)*
module load R

# initiate R in command line
R

```

*Note: for collaboration purposes, it's best for everyone to work with one version of R. Check what version is being used for the project you are working on. Some packages only work with some versions of R, so it's best to keep it consistent.

16.4.2 The ROAR Collab OnDemand Dashboard

The second way to test and edit code is to use the ROAR Collab OnDemand Dashboard, accessed by typing `login.sherlock.stanford.edu` into a web browser. You will be prompted to authenticate the way you would for any Stanford website. This is the best way to edit code for people who are not comfortable accessing & editing in base R in a Shell application.

You can test your code via the RStudio server on Sherlock. To access this, login to the Dashboard, then click on Interactive Apps in the menu bar and choose R Studio Server. Similar to the sdev node, you have to set various parameters for your session. Choose a version of R and set the time – max. 2 hours. You can play with the other configurations, but this is likely unnecessary, as you should not need huge computing power to test small amounts of code. Keep in mind the more computing power you request, the lower priority your request becomes. You will then wait for the resources to become available, and you will be able to click “Launch” when they are (if you don't mess with the CPU or GPU, this is usually less than 2 minutes). The screen that opens will look very similar to the RStudio on your local.

Do NOT use the RStudio Server's Terminal to install packages, set your R environment, and do everything else needed to configure Sherlock because you will likely need to re-do it for every session/project. It's best to use the Dashboard/RStudio Server if you are more comfortable testing & editing in RStudio rather than through base R in a Shell application.

16.4.3 Filepaths & configuration on ROAR Collab

In most cases, you will want to test that the file paths work correctly on ROAR Collab. You will likely need to add code to the configuration file in the project repository that specifies Sherlock-specific file paths. Here is an example:

```

# set sherlock-specific file paths
if(Sys.getenv("LMOD_SYSHOST")=="sherlock"){

```

```
sherlock_path = paste0(Sys.getenv("HOME"), "/malaria-project/")

data_path = paste0(sherlock_path, "data/")
results_path = paste0(sherlock_path, "results/")
}
```

16.5 Storage & group storage access

16.5.1 Individual storage

There are multiple places to store your files on Sherlock. Each user has their own `$HOME` directory as well as a `$SCRATCH` directory. These are directories that can be accessed via the command line once you’ve logged in to Sherlock:

```
cd $HOME
cd /home/users/USERNAME # Alternatively, use the full path

cd $SCRATCH
cd /scratch/users/USERNAME # Full path
```

You can also navigate to these using the File Explorer on Sherlock OnDemand.

`$HOME` has a volume quota of 15 GB. `$SCRATCH` has a volume quota of 100 TB, but files here get deleted 90 days after their last modification. Thus, use `$SCRATCH` for test files, exploratory analyses, and temporary storage. Use `$HOME` for long-term storage of important files and more finalized analyses.

You can read more about storage options on Sherlock [here](#).

16.5.2 Group storage

The lab also has a `$GROUP_HOME` and a `$GROUP_SCRATCH` to store files for collaborative use. `$GROUP_HOME` has a volume quota of 1 TB and infinite retention time, whereas `$GROUP_SCRATCH` has a volume quota of 100 TB and the same 90-day retention limit. You can access these via the command line or navigate to them using the File Explorer:

```
cd $GROUP_HOME
cd /home/groups/jadebc

cd $GROUP_SCRATCH
cd /scratch/groups/jadebc
```

However, saving files to group storage can be tricky. You can try using the `scp` command in the section “Moving files to Sherlock” to see if you have permission to add files to group directories. Read the next section to ensure any directories you create have the right permissions.

16.5.3 Folder permissions

Generally, when we put folders in `$GROUP_HOME` or `$GROUP_SCRATCH`, it is so that we can collaborate on an analysis within the research group, so multiple people need to be able to access the folders. If you create a new folder in `$GROUP_HOME` or `$GROUP_SCRATCH`, please check the folder's permissions to ensure that other group members are able to access its contents. To check the permissions of a folder, navigate to the level above it, and enter `ls -l`. You will see output like this:

```
drwxrwxrwx 2 jadebc jadebc 2204 Jun 17 13:12 myfolder
```

Please review this website to learn how to interpret the code on the left side of this output. The website also tells you how to change folder permissions. In order to ensure that all users and group members are able to access a folder's contents, you can use the following command:

```
chmod ugo+rw FOLDER_NAME
```

16.6 Running big jobs

Once your test scripts run successfully, you can submit an sbatch script for larger jobs. These are text files with a `.sh` suffix. Use a text editor like Sublime to create such a script. Documentation on sbatch options is available here. Here is an example of an sbatch script with the following options:

- `job-name=run_inc`: Job name that will show up in the Sherlock system
- `begin=now`: Requests to start the job as soon as the requested resources are available
- `dependency=singleton`: Jobs can begin after all previously launched jobs with the same name and user have ended.
- `mail-type=ALL`: Receive all types of email notification (e.g., when job starts, fails, ends)
- `cpus-per-task=16`: Request 16 processors per task. The default is one processor per task.
- `mem=64G`: Request 64 GB memory per node.
- `output=00-run_inc_log.out`: Create a log file called `00-run_inc_log.out` that contains information about the Slurm session
- `time=47:59:00`: Set maximum run time to 47 hours and 59 minutes. If you don't include this option, Sherlock will automatically exit scripts after 2 hours of run time.

The file `analysis.out` will contain the log file for the R script `analysis.R`.

```
#!/bin/bash

#SBATCH --job-name=run_inc
#SBATCH --begin=now
```

```
#SBATCH --dependency=singleton
#SBATCH --mail-type=ALL
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --mem=64G
#SBATCH --output=00-run_inc_log.out
#SBATCH --time=47:59:00

cd $HOME/malaria-code-repo/2-analysis/

module purge

# load R version 4.0.2 (required for certain packages)
module load R/4.0.2

# load gcc, a C++ compiler (required for certain packages)
module load gcc/10

# load software required for spatial analyses in R
module load physics gdal
module load physics proj

R CMD BATCH --no-save analysis.R analysis.out
```

To submit this job, save the code in the chunk above in a script called `myjob.sh` and then enter the following command into terminal:

```
sbatch myjob.sh
```

To check on the status of your job, enter the following code into terminal:

```
squeue -u $USERNAME
```


Chapter 17

Checklists

by Jade Benjamin-Chung

17.1 Pre-analysis plan checklist

- Brief background on the study (a condensed version of the introduction section of the paper)
- Hypotheses / objectives
- Study design
- Description of data
- Definition of outcomes
- Definition of interventions / exposures
- Definition of covariates
- Statistical power calculation
- Statistical model description
- Covariate selection / screening
- Standard error estimation method
- Missing data analysis
- Assessment of effect modification / subgroup analyses
- Sensitivity analyses
- Negative control analyses

17.2 Code checklist

- Does the script run without errors?
- Is code self-contained within repo and/or associated Box folder?
- Is all commented out code / remarks removed?
- Does the header accurately describe the process completed in the script?
- Is the script pushed to its github repository?

- Does the code adhere to the coding style guide?
- Are all warnings ignorable? Should any warnings be intentionally suppressed or addressed?

17.3 Manuscript checklist

This is adapted in part from this article.

- Have you completed the relevant reporting checklist, if applicable? (Collection of checklists)
- Are the study results within the manuscript replicable (i.e., if you rerun the code in the study’s repository, the tables and figures will be exactly replicated?)
- Is a target journal selected?
- Is the title declarative, in other words, does it state the object/findings rather than suggest them?
- Is the word count of the manuscript close to the target journal’s allowance?
- Does the manuscript adhere to the formatting guide of the target journal?
- Does the manuscript use a consistent voice (passive or active – usually active is preferred ... pun intended)?
- Is each figure and table (including supplementary material) referenced in the main text?
- Is there a caption for each figure and table (including supplementary material)?
- Are tables/figures and supplementary material numbered in accordance with their appearance in the main text?
- Does the text use past tense if it is reporting research findings or future tense if it is a study protocol?
- Does the text avoid subjective wording (e.g., “interesting”, “dramatic”)?
- Does the text use minimal abbreviations, and are all abbreviations defined at first use?
- Does the text avoid directionless words? (e.g., instead of writing, ‘Precipitation influences disease risk’, write, ‘Precipitation was associated with increased disease risk’).
- Does the text avoid making causal claims that are not supported by the study design? Be careful about the words “effect”, “increase”, and “decrease”, which are often interpreted as causal.
- Does the text avoid describing results with the word “significant”, which can easily be confused with statistical significance? (see references on this topic here)
- Have you drafted author contributions? Do they follow the CRediT Taxonomy for author contributions?

17.4 Figure checklist

- Are the x-axis and y-axis labeled?
- If the figure includes panels, is each panel labeled?
- Are there sufficient numerical / text labels and breaks on the x-axis and y-axis?
- Is the font size appropriate (i.e., large enough to read, not so large that it distracts from the data presented in the figure?)
- Are the colors used colorblind friendly? See a colorblind-friendly palette [here](#), a neat palette generator with colorblind options [here](#), and an article on why this matters [here](#)
- Are colors/shapes/line types defined in a legend?
- Are the legends and other labels easy to understand with minimal abbreviations?
- If there is overplotting, is transparency used to show overlapping data?
- Are 95% confidence intervals or other measures of precision shown, if applicable?

Chapter 18

Resources

by Jade Benjamin-Chung, Kunal Mishra, and Jess Grembi

18.1 Resources for R

18.1.1 For Beginners

- Introduction to R
- R Learning Path: from beginner to expert in 7 steps
- Swirl
- Stanford R Bootcamp - Summer 2020
- R Style Guide

18.1.2 For Intermediate/Advanced Users

- dplyr and tidyr cheat sheet
- ggplot cheat sheet
- data table cheat sheet
- RMarkdown cheat sheet
- Hadley Wickham's R Style Guide
- Jade's R-for-epi course
- Tidy Eval in 5 Minutes (video)
- Tidy Evaluation (e-book)
- Data Frame Columns as Arguments to Dplyr Functions (blog)
- Standard Evaluation for `*__join` (stackoverflow)
- Programming with dplyr (package vignette)

18.2 Resources for Git & Github Specifically tailored for R users

- Happy Git and GitHub for the useR
- Git and Github in RStudio
- Setting your GitHub credentials

18.3 Scientific figures

- Ten Simple Rules for Better Figures

18.4 Writing

- Tips on how to write a great science paper
- ICMJE Definition of authorship
- Nature article on elements of style for scientific writing
- The Pathway to Publishing: A Guide to Quantitative Writing in the Health Sciences
- Secret, actionable writing tips

18.5 Presentations

- How to tell a compelling story in scientific presentations
- How to give a killer narratively-driven scientific talk
- How to make a better poster
- How to make an even better poster

18.6 Professional advice

- Professional advice, especially for your first job

18.7 Funding

- Building Your Funding Train

18.8 Ethics and global health research

- Global Code of Conduct For Research in Resource-Poor Settings
- Who is a global health expert? Advice for aspiring global health experts
- Transforming Global Health Partnerships